

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Big Data Analytics

Final Project

Distributed Multi-Model Analytics for E-commerce Data

with MongoDB, HBase and Apache Spark

Document and wide-column data models, batch processing, Spark SQL, an integrated customer lifetime value analysis, visualisation and recommendations for 100,000 sessions and 30,000 transactions of an online shop

Student

No.	Full name	Student ID
1	Erneste UWIMPUHWE	101414

Source code and README on GitHub

GitHub Repository

`https://github.com/erneste-uwimpuhwe/101414_BDA_FINAL_EXAM_PROJECT_SUBMISSION`

Lecturer: **Temitope Ogutande**

Table of Contents

Introduction . . . **3**
- Aim and questions . . . 3
- Approach and tools . . . 3
- Main findings . . . 3
- How the report is organised . . . 4

1 Generate and inspect the data . . . **5**
- 1.1 Prepare the workspace . . . 5
- 1.2 Generate the dataset . . . 6

2 Build MongoDB . . . **8**
- 2.1 Design the document model . . . 8
- 2.2 Start MongoDB and load the data . . . 8
- 2.3 Run three aggregation pipelines . . . 10

3 Build HBase . . . **12**
- 3.1 Start HBase in Docker . . . 12
- 3.2 Define the two tables . . . 13
- 3.3 Load and query the data . . . 14

4 Process the data with Spark . . . **16**
- 4.1 Prepare Spark and clean the data . . . 16
- 4.2 Products viewed together, and cohorts . . . 17
- 4.3 Analyse with Spark SQL . . . 18

5 Integrate MongoDB, HBase and Spark . . . **21**
- 5.1 Extract purchases and profiles from MongoDB . . . 21
- 5.2 Extract session activity from HBase . . . 21
- 5.3 Join and estimate lifetime value in Spark . . . 22

6 Visualise and interpret the results . . . **25**
- 6.1 Conversion funnel . . . 25
- 6.2 Sales, customer value and categories . . . 26
- 6.3 Business insights and actions . . . 27

7 Scale and defend the design . . . **28**
- 7.1 Scalability . . . 28
- 7.2 Technology selection . . . 28
- 7.3 Limitations and future work . . . 28

8 Submit reproducible work . . . **29**
- 8.1 Repository . . . 29
- 8.2 Reproduce the results . . . 29
- 8.3 Academic integrity . . . 30

Introduction

An online shop wants its analysts to understand how customers browse, buy and are worth over time, and to choose the right data technology for each question. This report documents how we stored the same e-commerce data in MongoDB and HBase, processed it with Apache Spark, combined the stores in one customer lifetime value analysis, and turned the results into charts and recommendations.

The data come from the supplied generator `dataset_generator.py`. The supplied file is a Colab notebook export, so its single code cell was extracted without changing the generation logic; only the target volumes were set to 100,000 sessions and 30,000 transactions so that the project runs on an 8 GB laptop. The data hold 10,000 users, 5,000 products, 25 categories, 100,000 sessions and 30,000 transactions over about 90 days (August to October 2026). Every result was produced by our own programs.

Aim and questions

The project answers five practical questions:

- How should products, users, transactions and sessions be modelled in MongoDB, and the session time series in HBase? (Sections 2 and 3)
- Can Spark clean the data, find products viewed or bought together, and describe cohorts and aggregates in SQL? (Section 4)
- Which customers are the most valuable, and does browsing engagement help to identify them? (Section 5)
- What do the funnel, sales, category and customer charts show, and what should the business do? (Section 6)
- How does the design scale, why was each technology chosen, and what are its limits? (Sections 7 and 8)

Approach and tools

Everything ran on one laptop (Windows 11 Pro, 8,035 MB of RAM). MongoDB 8.2.4 runs as its own instance, HBase 2.1.3 runs in Docker, and Apache Spark 4.2.0 runs in local mode with Java 17 and Python 3.13.6 (pymongo 4.19.0, happybase 1.3.0, pyspark 4.2.0). Because memory is limited, only one heavy tool ran at a time. Repeated evidence is referenced instead of repeated, and complete outputs and code are kept in the GitHub repository.

Main findings

- **Same data, same answers.** Category revenue and top-product units are identical in MongoDB and Spark, and the embedded `purchase_summary` matches the aggregation pipeline for every user (0 mismatches).
- **Revenue is not profit.** The highest-revenue category ranks 15th in estimated profit; the 17th by revenue ranks first.
- **Funnel.** Only 20.4% of sessions end in a purchase; the largest loss is between cart and checkout, where 44.8% of carts continue.
- **HBase row key.** A `user_id` plus reversed-timestamp key returns a user's newest sessions first with no sorting.

- **Customer value.** Platinum customers are worth 6.2 times Bronze customers over 12 months but browse only 1.12 times as much; 76 heavy browsers have never bought.
- **Recommendation.** Use MongoDB for documents and aggregation, HBase for key-ordered session scans, and Spark for cleaning, SQL and the join across systems.

How the report is organised

Section	Content
1	Workspace, versions and settings; generation and inspection of the dataset
2	MongoDB document model, load, indexes and three aggregation pipelines
3	HBase in Docker, row-key and column-family design, load and queries
4	Spark: cleaning, products viewed or bought together, cohorts and Spark SQL
5	Integrated customer lifetime value analysis across MongoDB, HBase and Spark
6	Funnel, sales, customer-value and category charts, with insights and actions
7	Scalability, technology selection, limitations and future work
8	Repository, reproduction steps and verification

1 Generate and inspect the data

1.1 Prepare the workspace

All work was done on one laptop under Windows 11 Pro (build 26300) with 8,035 MB of RAM. The project lives in `D:\BDA_Project` with the folders `data`, `mongo`, `hbase`, `spark`, `integration`, `visuals` and `screenshots` (Figure 1.1-1), so the generated data, the MongoDB files and the HBase mount are all stored on drive D:. A script `env.bat` pins Java 17, Spark and the Hadoop utilities for Windows and activates the Python virtual environment, so every command window starts with the same settings (Figure 1.1-2).

Item	Value
Operating system	Microsoft Windows 11 Pro, version 10.0.26300, build 26300
Memory	8,035 MB of RAM; WSL memory for Docker capped at 3 GB
Python and packages	Python 3.13.6 (virtual environment); pymongo 4.19.0, happybase 1.3.0, pyspark 4.2.0, pandas 2.3.3, numpy 2.5.4, ijson 3.6.0, matplotlib 3.11.2, faker 40.44.0
Java	OpenJDK Temurin 17.0.20 (LTS), selected by `env.bat`
Apache Spark	4.2.0 with Scala 2.13.18; local mode, 2 cores, 2 GB driver memory, 8 shuffle partitions
MongoDB	8.2.4 standalone instance on port 27018; data in `D:\BDA_Project\mongo\data`; WiredTiger cache 0.5 GB
HBase	2.1.3 in Docker (`harisekhon/hbase:2.1`), 1.8 GB memory limit; Thrift port 9090, Master UI port 16010
Docker	Docker 29.8.1

Table 1-1: Recorded environment

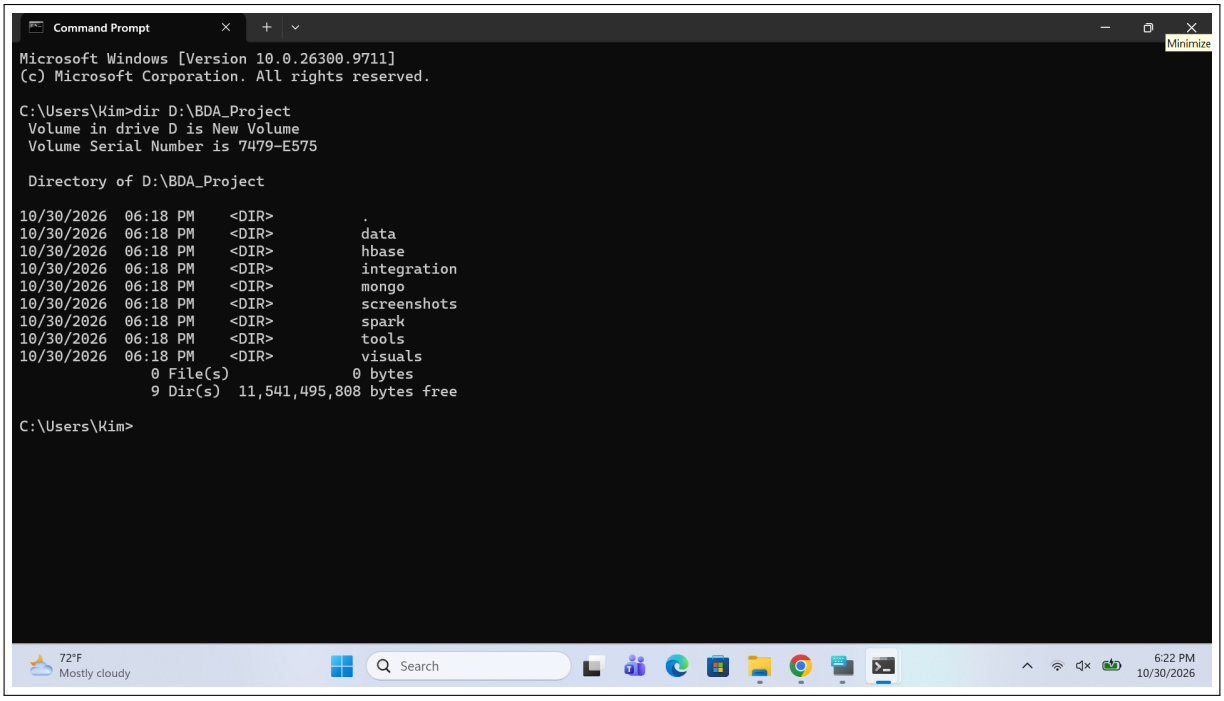


Figure 1.1-1: Project folders on drive D: (`data`, `hbase`, `integration`, `mongo`, `screenshots`, `spark`, `tools`, `visuals`)

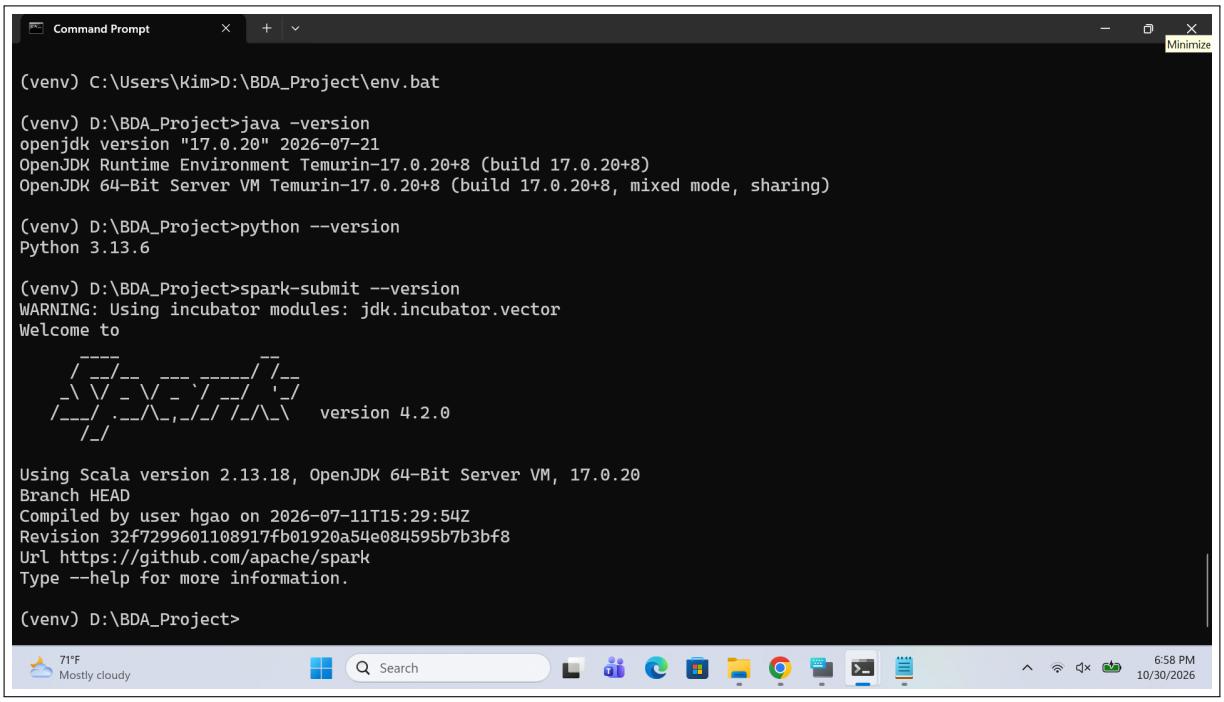


Figure 1.1-2: Java 17 (Temurin 17.0.20), Python 3.13.6 and Spark 4.2.0 with Scala 2.13.18, all started through `env.bat`

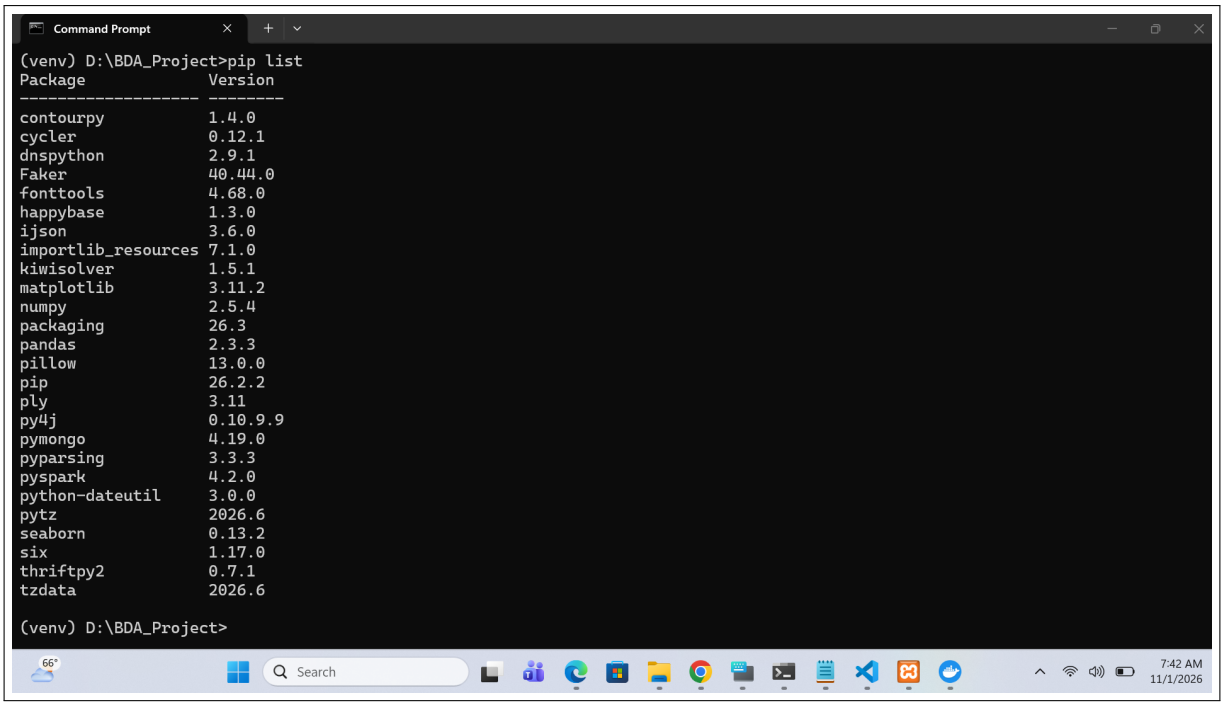


Figure 1.1-3: Python packages installed in the project's virtual environment

1.2 Generate the dataset

The generator builds 25 categories (each with 3 to 5 subcategories and a profit margin), 5,000 products with price history, 10,000 users, and then sessions and transactions together: every session follows realistic page transitions (home, category, product, cart, checkout), carts contain only viewed products, and stock is reduced at each purchase so that sales cannot exceed inventory. With the targets set to 100,000 sessions and 30,000 transactions it finished with both targets met (Figure 1.2-1) and wrote the five groups of JSON files, 204.5 MB in total, of which `sessions_0.json` is 188 MB (Figure 1.2-2).

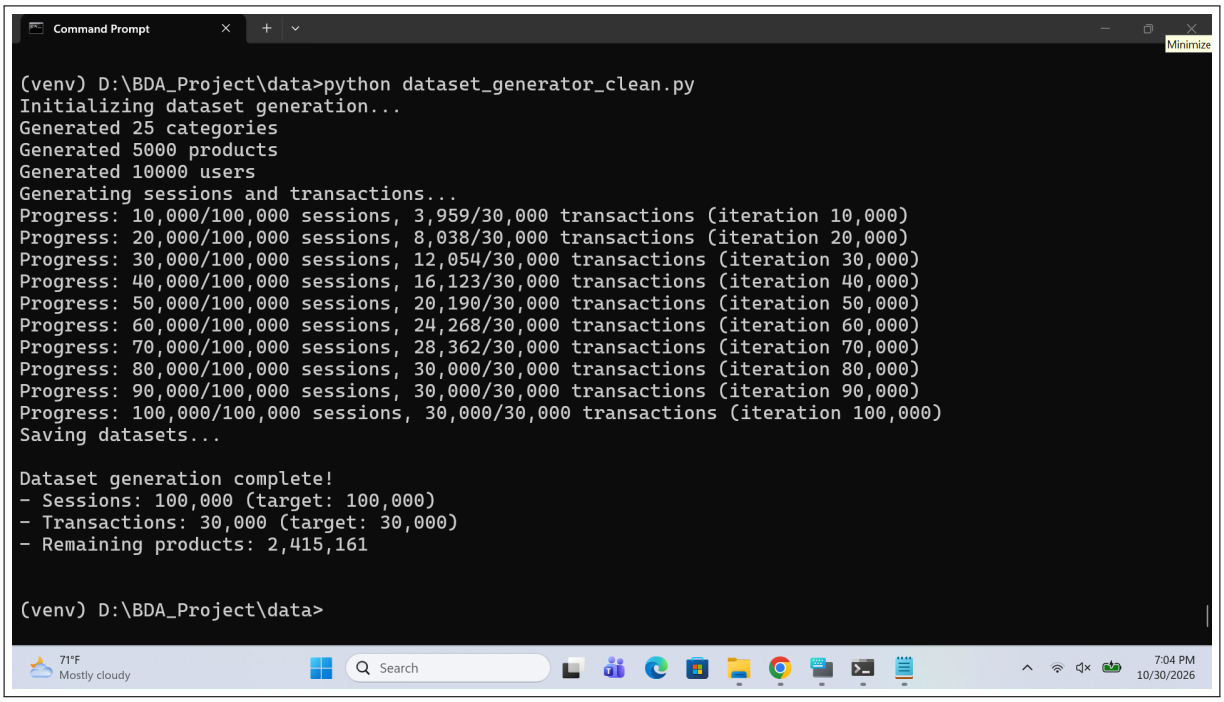


Figure 1.2-1: Dataset generation completed: 100,000 sessions and 30,000 transactions, 25 categories, 5,000 products, 10,000 users

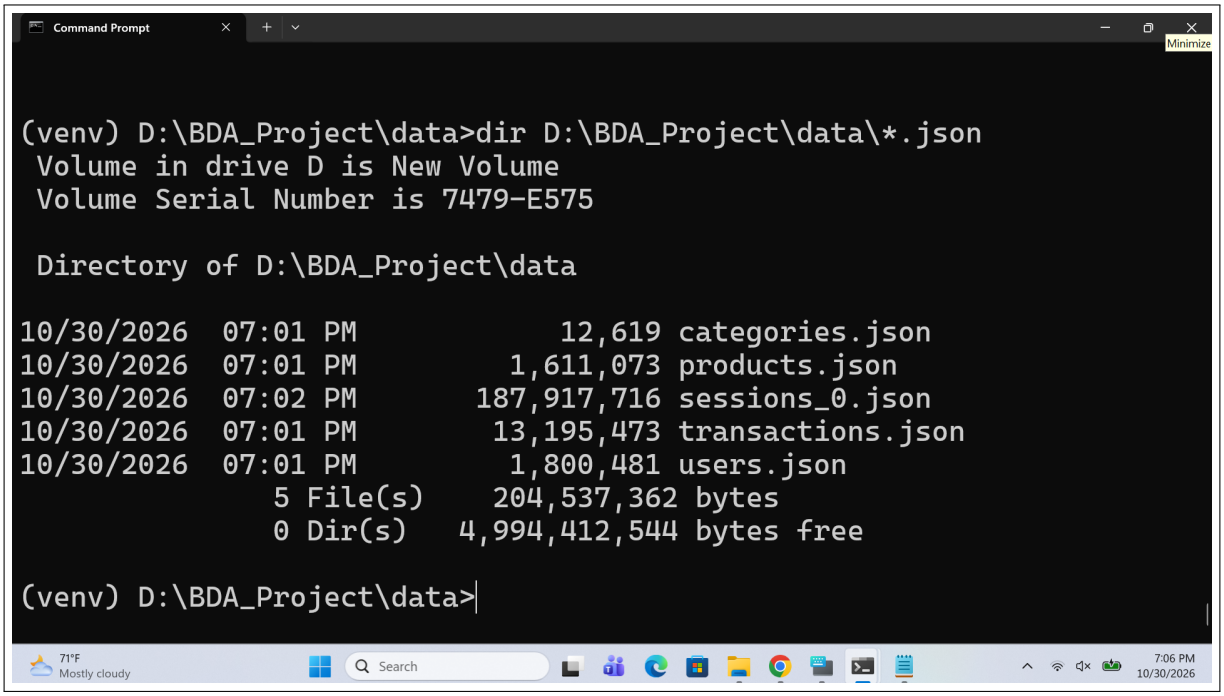


Figure 1.2-2: The generated JSON files on drive D: with their sizes (204,537,362 bytes in total)

Two properties of the generator matter for later analysis and are stated once here. Products carry `category_id` but no `subcategory_id`, although the project brief's sample shows one, so analysis is at category level. The generator also creates extra transactions that are not linked to any session, so a product can have more purchases than views and a user with orders can have no converted session.

2 Build MongoDB

2.1 Design the document model

The model follows the questions to be asked (Table 2-1). Data that is always read together is embedded, so no join is needed at query time. Indexes were created on `transactions.user_id`, `transactions.timestamp`, `transactions.items.product_id`, on `sessions` by `user_id` and `start_time`, and on `products.category_id`.

Collection	Documents	Design	Reason
products	5,000	Category embedded (id, name, average profit margin of its subcategories); price history as an array	Category revenue and margin need no second lookup
users	10,000	Profile plus embedded `purchase_summary` (orders, total spent, last purchase)	Read often; checked against the transactions
transactions	30,000	Line items embedded as an `items` array	Items are always read with their order
sessions	100,000	Page views and cart nested in one document per session	The whole journey is one read
categories	25	Subcategories with profit margins as an array	Reference data

Table 2-1: MongoDB collections and the reason for each design

2.2 Start MongoDB and load the data

A dedicated `mongod` instance is started on port 27018 with its data on drive D: and a 0.5 GB cache, so it does not touch the MongoDB service that was already installed (Figure 2.2-1). The loader `load_mongo.py` converts date strings to dates, embeds the category in each product, computes each user's purchase summary from the transactions, and streams the 100,000 sessions in batches of 5,000 with `ijson`, so the 188 MB session file is never held in memory. It loaded 25 categories, 5,000 products, 30,000 transactions, 10,000 users and 100,000 sessions and then created the indexes (Figure 2.2-2). One user document shows the embedded summary: `user_000000` has 2 orders and 3,934.69 spent (Figure 2.2-3).

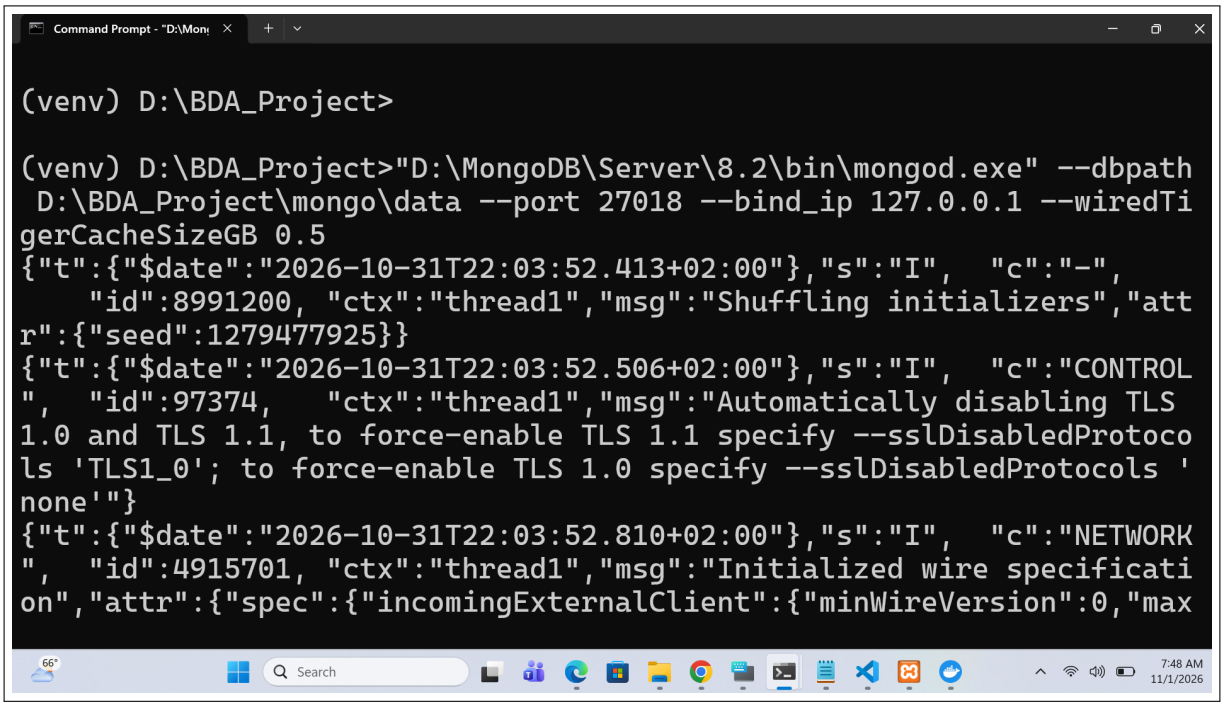


Figure 2.2-1: `mongod` started with data path `D:\BDA_Project\mongo\data`, port 27018 and a 0.5 GB cache

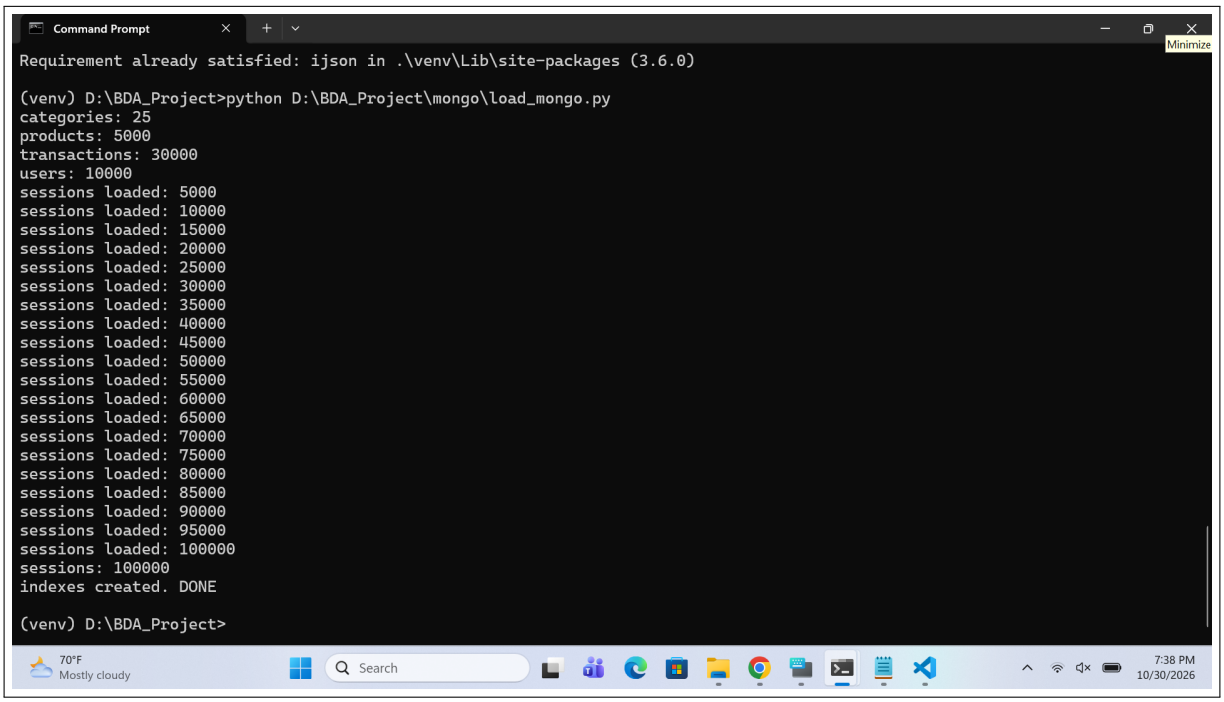


Figure 2.2-2: `load_mongo.py` output: five collections loaded and indexes created

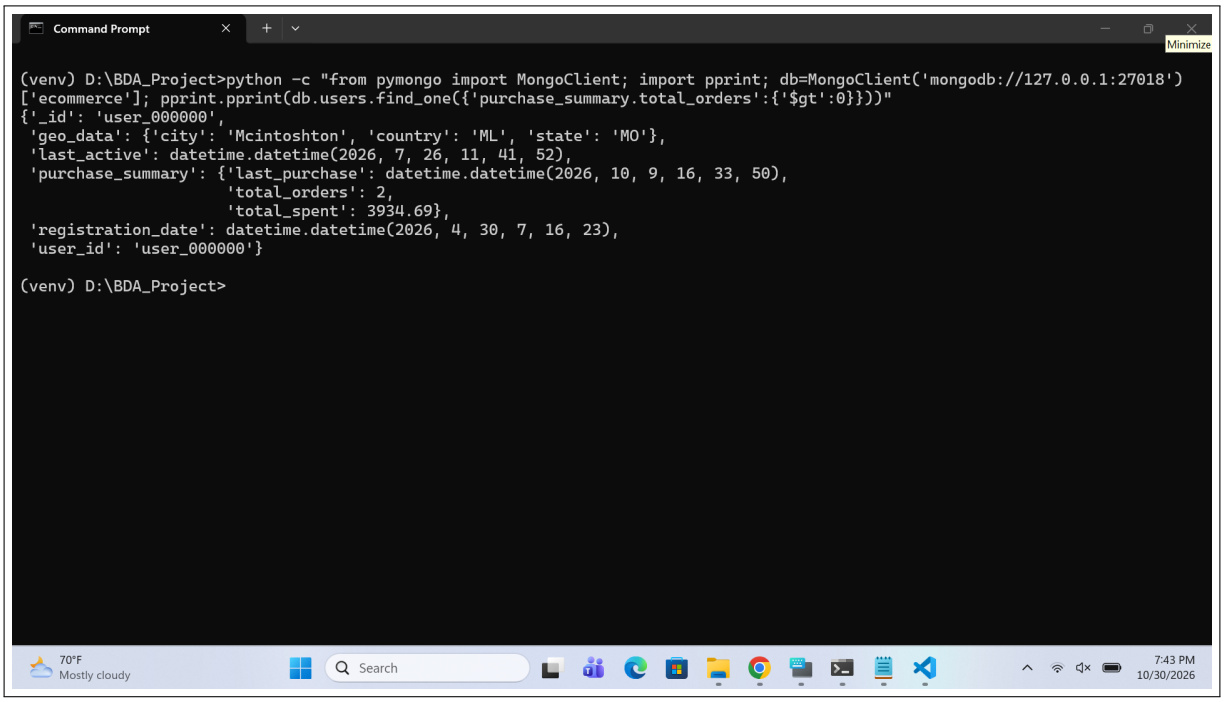


Figure 2.2-3: A user document with the embedded `purchase_summary`

2.3 Run three aggregation pipelines

The three pipelines run inside MongoDB (`mongo_aggregations.py`). **Pipeline 1, product popularity**, unwinds the `items` array, groups by product, joins the catalog with `$lookup` and sorts by units sold. The top seller is `prod_01476` with 61 units over 26 orders (22,623.07 revenue), and the top ten lie between 55 and 61 units, so there is no runaway best seller (Figure 2.3-1).

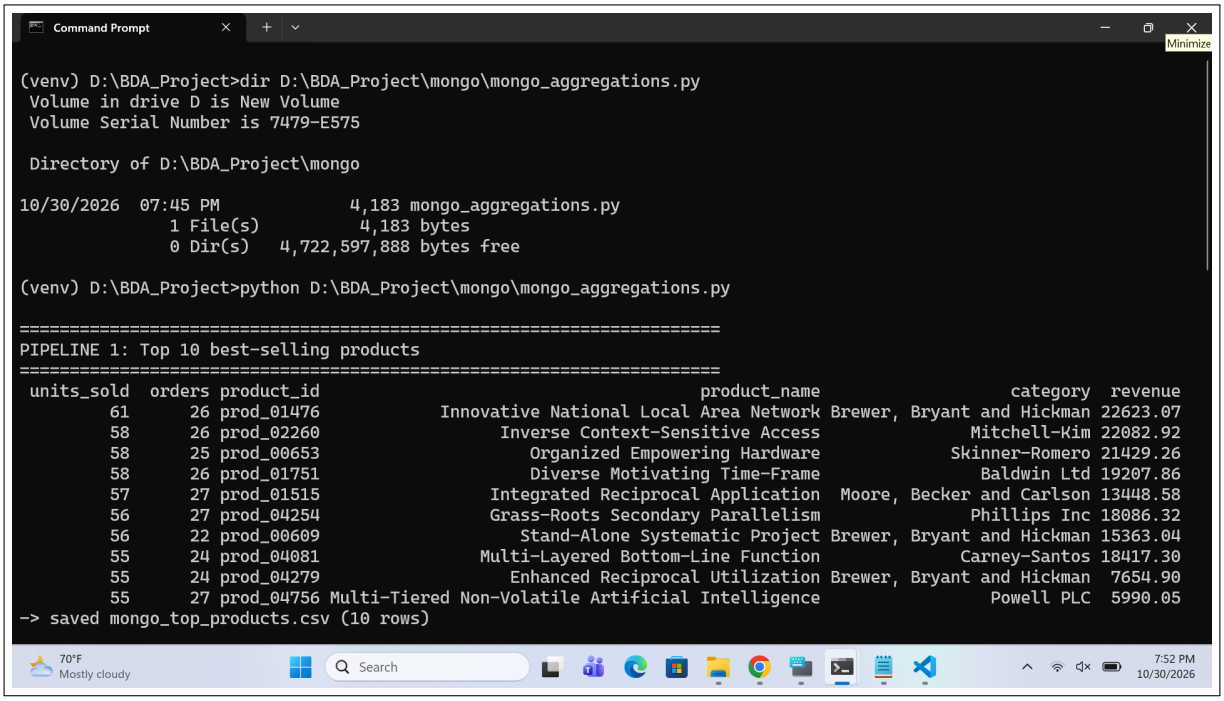


Figure 2.3-1: Pipeline 1: the ten best-selling products by units sold

Pipeline 2, revenue by category, multiplies each line item by the profit margin embedded in the product. Kelly-Santiago has the highest revenue (1,598,265.70) but an estimated profit of 329,242.73, whereas

Rodriguez-Graham has 1,242,448.47 revenue and the highest profit, 451,008.79 (Figure 2.3-2).

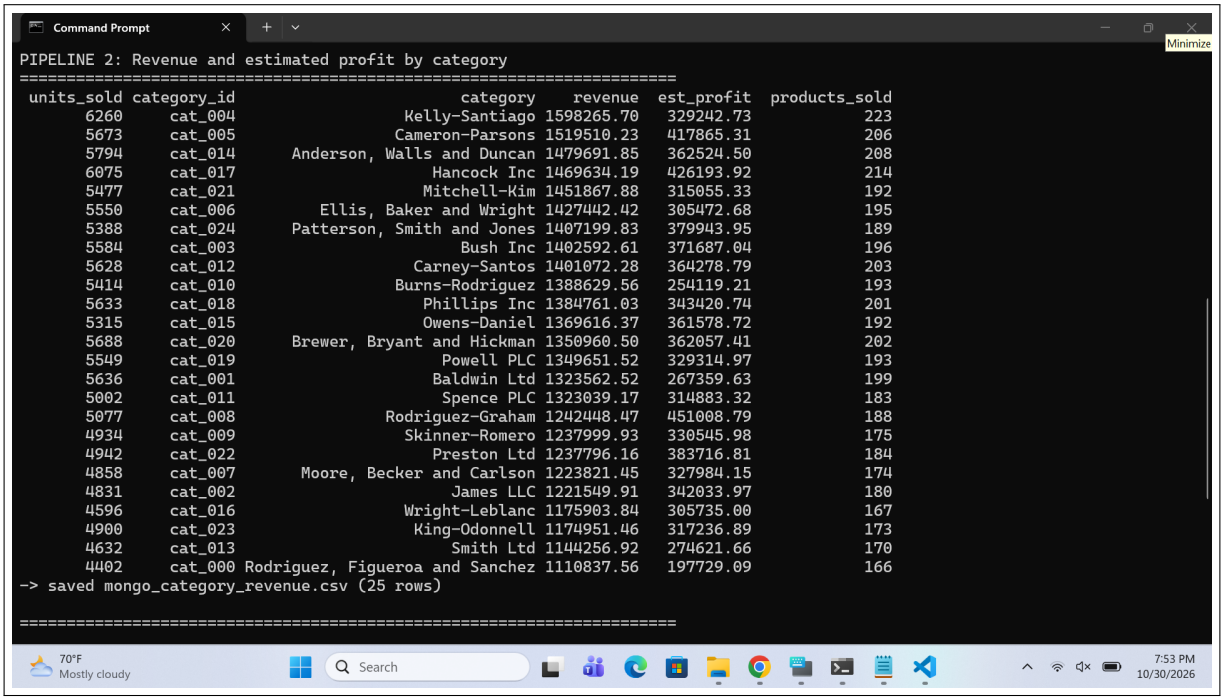


Figure 2.3-2: Pipeline 2: revenue, estimated profit and units by category

Pipeline 3, segmentation by purchase frequency, uses the embedded summary: 496 users have not bought, 3,730 are occasional (1 to 2 orders), 4,962 regular (3 to 5) and 812 loyal (6 or more). Regular buyers produce 20.3 of the 32.6 million total spend, about 62% (Figure 2.3-3). The dataset has no age field, so purchase frequency is used instead of age groups.

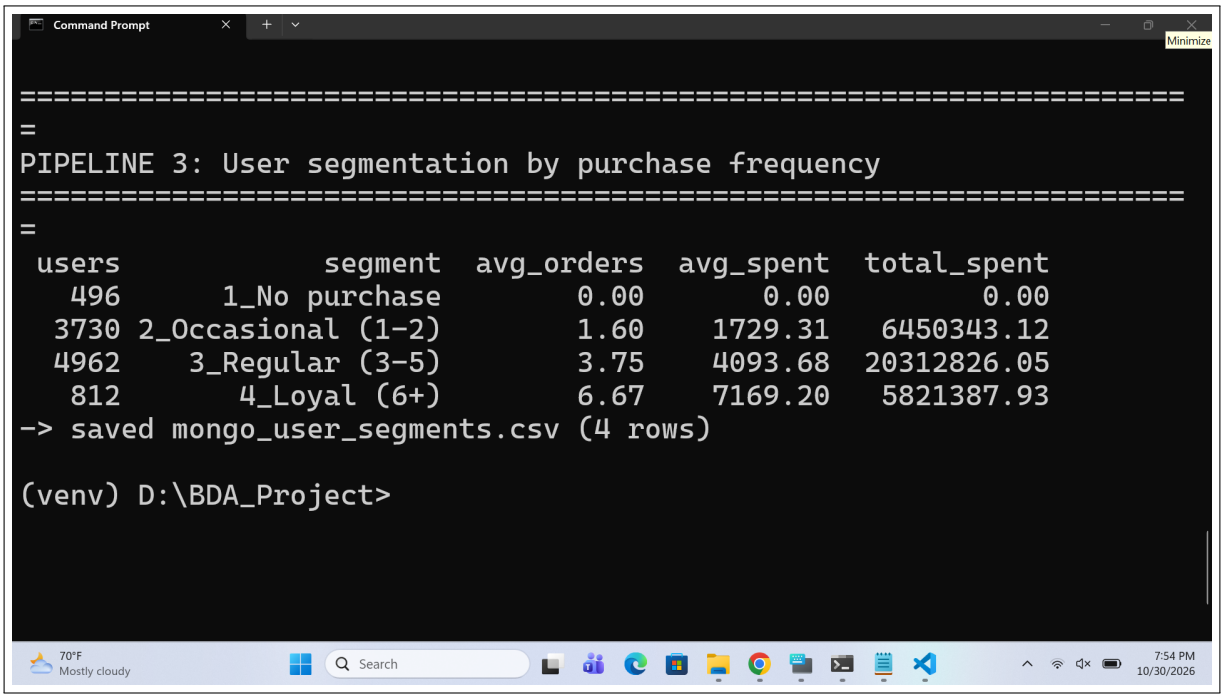


Figure 2.3-3: Pipeline 3: users, average orders and spend by purchase-frequency segment

3 Build HBase

3.1 Start HBase in Docker

HBase 2.1.3 runs as one container (image `harisekhon/hbase:2.1`) limited to 1.8 GB of memory. Ports 2181, 9090 (Thrift, used by Python) and 16010 (web interface) are published, and the project folder `D:\BDA_Project\hbase` is mounted as `/work`, so scripts and data stay on drive D:. The shell reports one active master and one server (Figure 3.1-1), and the Master web page shows one region server (Figure 3.1-2); the two regions listed there are HBase's own system tables, before the project tables were created.

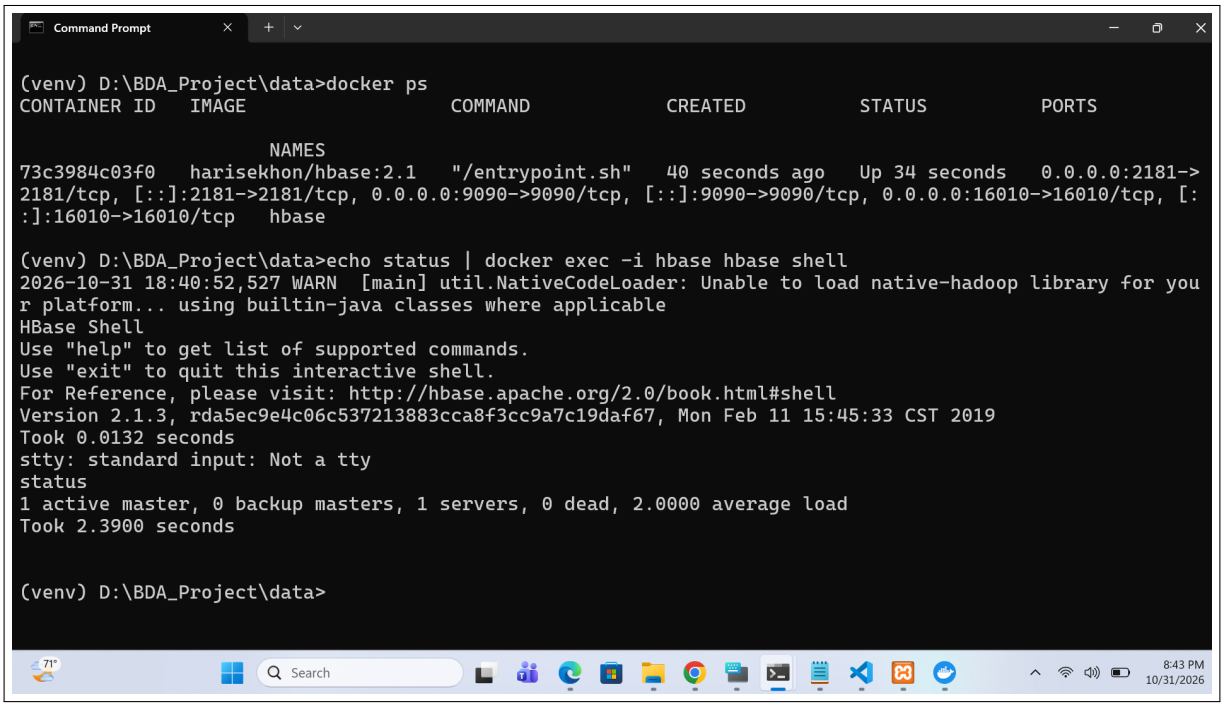


Figure 3.1-1: The `hbase` container is up; the shell reports 1 active master, 0 backup masters and 1 server

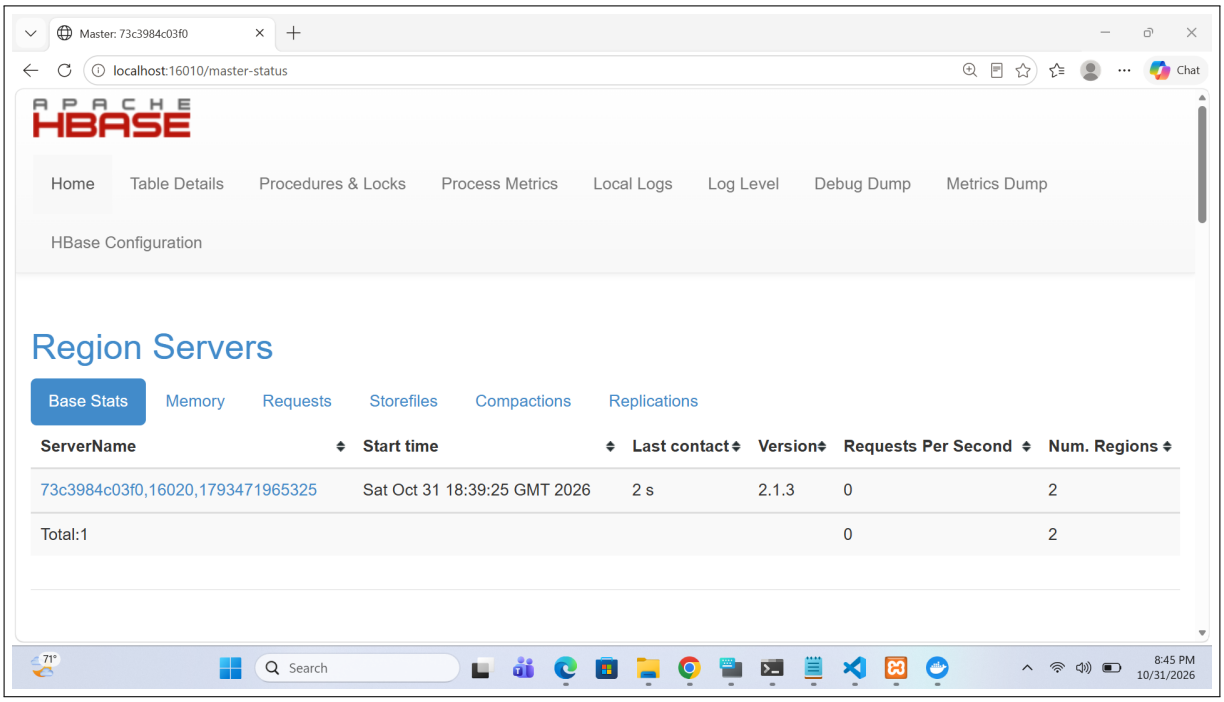


Figure 3.1-2: HBase Master web interface at `localhost:16010` with one region server (version 2.1.3)

3.2 Define the two tables

The tables are built for the two access patterns that suit a wide-column store (Table 3-1). In `user_sessions` the row key is the user id followed by `9999999999` minus the session's epoch seconds, zero-padded. Because HBase sorts rows by bytes, a prefix scan on one user returns that user's sessions newest first with no sorting, and the user prefix spreads writes across users instead of piling them on the latest time. Columns read together share a family: `info` for session facts and geography, `device`, and `activity` for page views, cart and products. In `product_daily_metrics` one row per product per day holds views, cart additions and purchases, so a product's history is a short range scan. Both tables keep one version per cell (Figure 3.2-1).

Table	Row key	Column families	Purpose
`user_sessions`	`user_id#(9999999999 - epoch seconds)`	`info`, `device`, `activity`	One user's sessions, newest first
`product_daily_metrics`	`product_id#yyyymmdd`	`m` (views, cart_adds, purchases)	Daily time series per product

Table 3-1: HBase tables

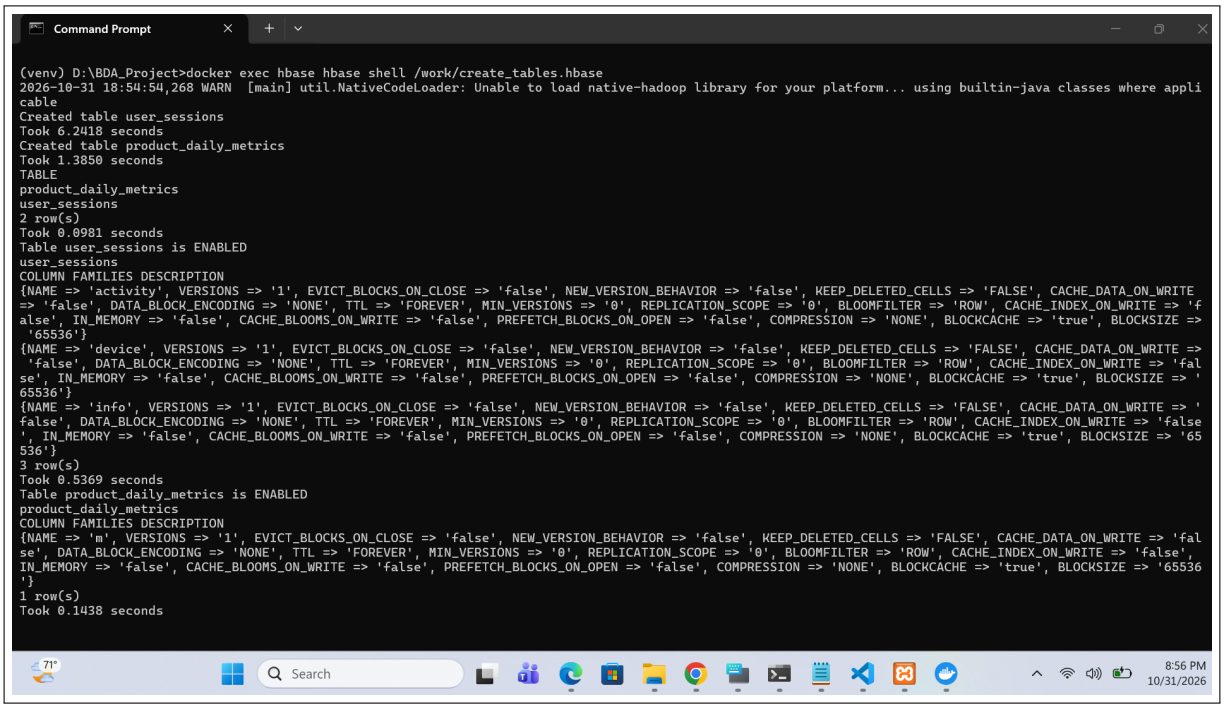


Figure 3.2-1: `create_tables.hbase`: both tables created and enabled, with families `activity`, `device`, `info` and `m`

3.3 Load and query the data

`hbase_load.py` streams the sessions, writes each one as a row of 15 columns through the Thrift server in batches of 2,000, and counts product views (product-detail page views), cart additions and purchases per product and day. It loaded 100,000 session rows and 255,199 product-day rows (Figure 3.3-1).

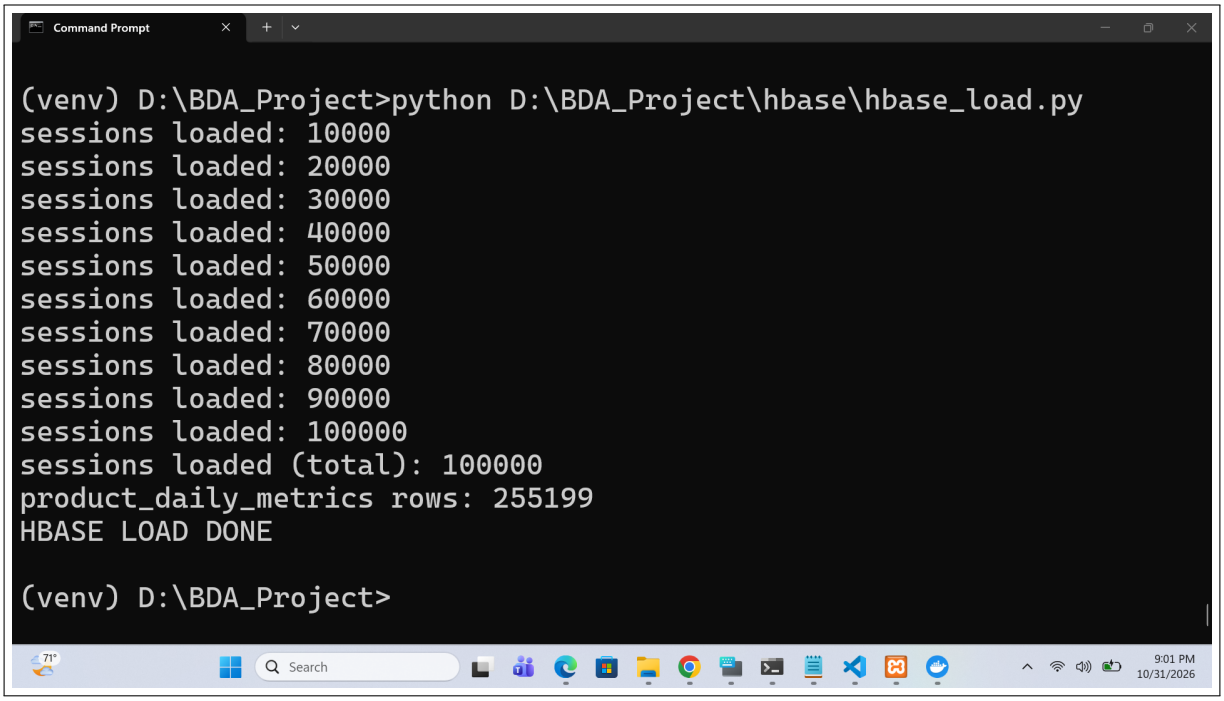


Figure 3.3-1: `hbase_load.py` output: 100,000 session rows and 255,199 product-day rows loaded

The queries (Figure 3.3-2) are a prefix scan of the five most recent sessions of `user_000000` (20 October, 17 October, 7 October, 29 September and 12 August, newest first), a time-range scan for October 2026 that returns 3 of the user's 7 sessions, and the daily metrics of `prod_01476`. On 4 August that product shows 2

purchases against 1 view, because some generated transactions have no session.

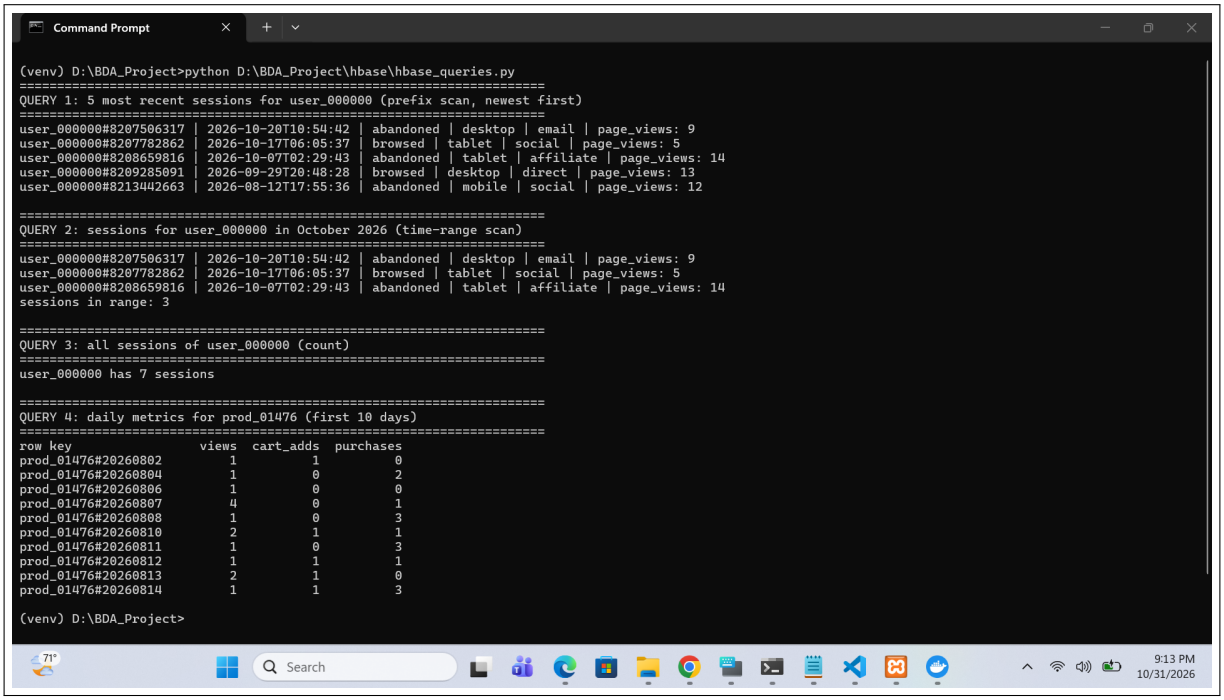


Figure 3.3-2: Python queries: prefix scan, October range scan, session count and daily product metrics

The same data read in the HBase shell (Figure 3.3-3) returns only the requested columns of three session rows and five daily-metric rows, and a `get` returns one row (`prod_01476#20260807`: 4 views, 0 cart additions, 1 purchase).

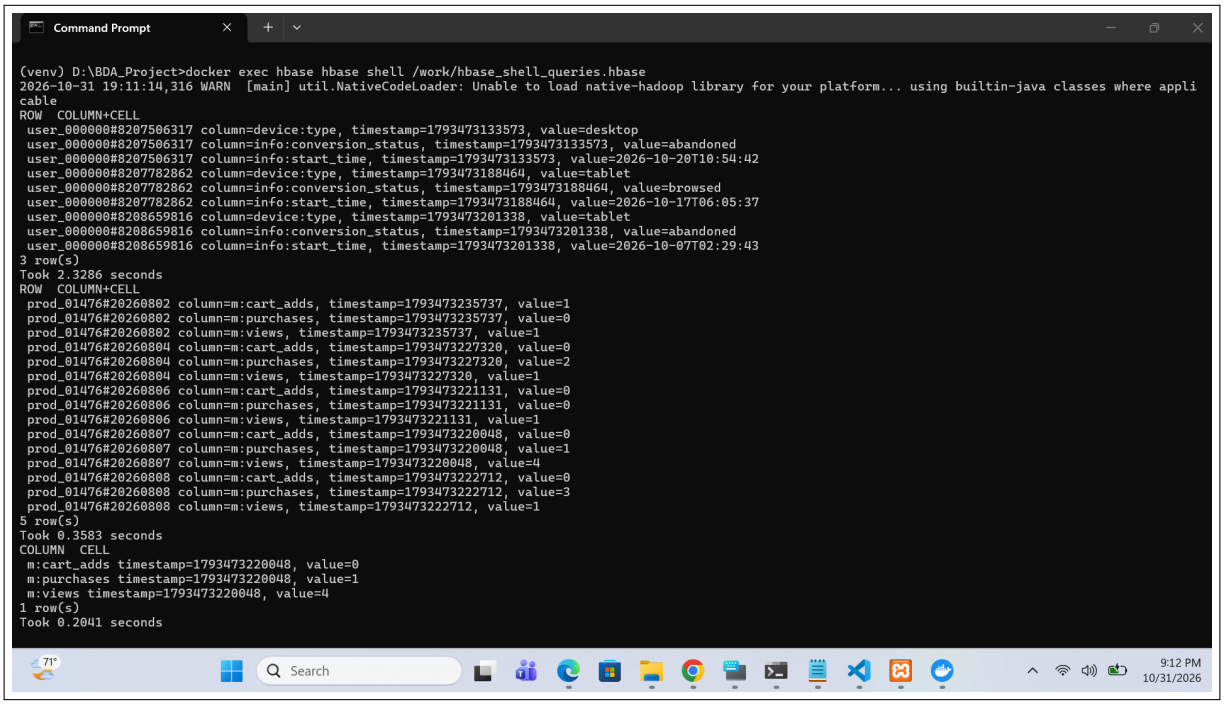


Figure 3.3-3: HBase shell: scans of `user_sessions` and `product_daily_metrics`, and a `get` of one row

4 Process the data with Spark

4.1 Prepare Spark and clean the data

Apache Spark 4.2.0 runs in local mode with two cores and a 2 GB driver (Figure 4.1-1). The 188 MB session file is a single JSON array, which Spark would read in one task, so `prep_sessions_jsonl.py` first converts it to JSON Lines (one session per line, 100,000 lines) that Spark reads in parallel. The cleaning job `spark_clean.py` then reads all five datasets with explicit schemas; the cart is declared as a map because its keys are product ids, otherwise Spark would infer about 5,000 columns.

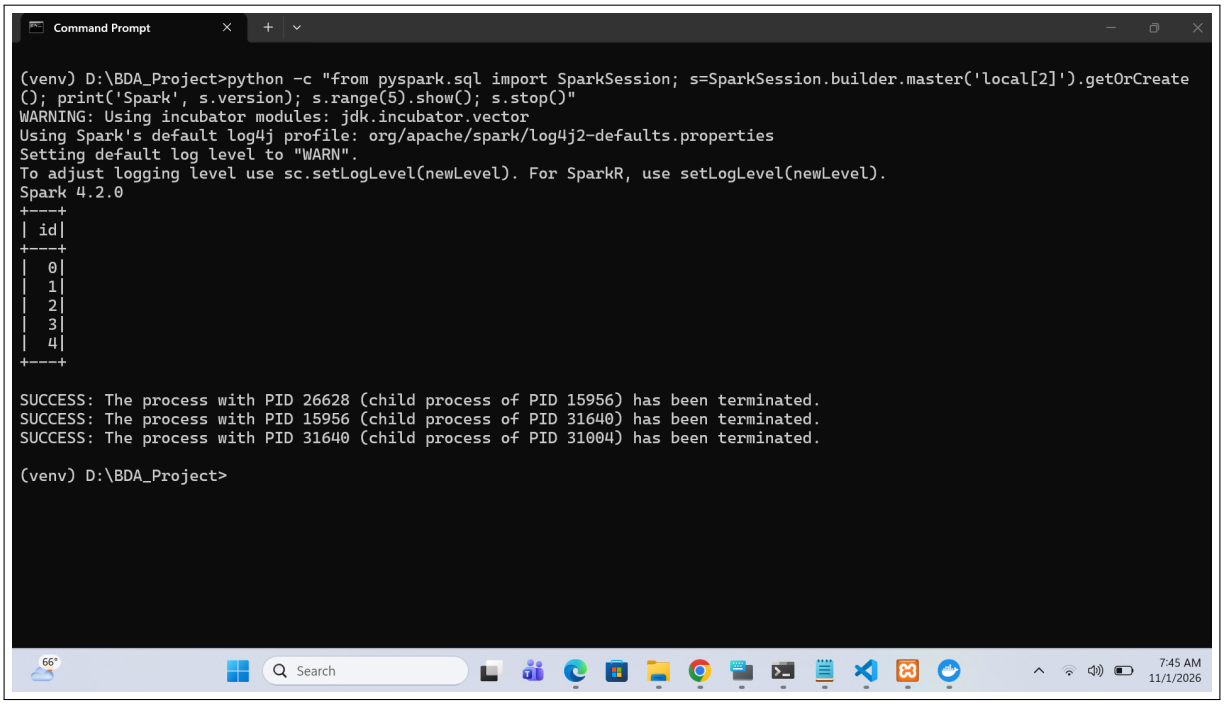


Figure 4.1-1: Spark 4.2.0 session test in local mode

The job applies these rules: drop null keys, remove duplicate ids, cast timestamps safely (a bad value becomes NULL instead of failing the job), reject non-positive prices, negative stock, non-positive durations and sessions that end before they start, normalise case, and flatten nested arrays into `page_views`, `cart_items`, `transaction_items`, `subcategories` and `price_history` tables stored as Parquet. The data-quality report (Figure 4.1-2) shows 10,000 users, 25 categories, 5,000 products, 30,000 transactions and 100,000 sessions both raw and clean, with 0 rows rejected and 0 transactions where subtotal minus discount differs from the total. Zero rejections show that the generated data satisfy the rules; the rules were still applied.

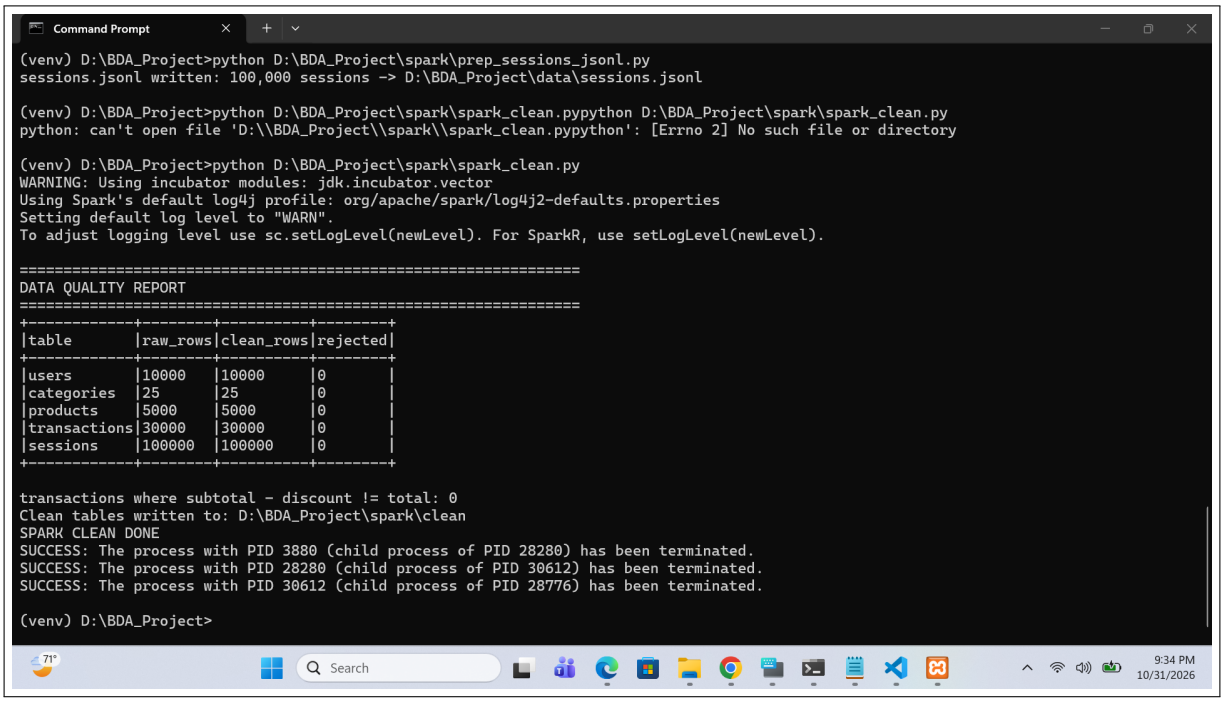


Figure 4.1-2: Data-quality report: raw rows, clean rows and rejected rows per table

4.2 Products viewed together, and cohorts

`spark_analysis.py` counts product pairs that appear in the same session (98,733 sessions viewed products) and in the same order (30,000 orders) with a self-join, and reports pair count, confidence and lift. The most frequent viewed-together pairs co-occur only 3 to 4 times, with lift between 57 and 111; the bought-together pairs co-occur twice, with lift between 476 and 750 (Figure 4.2-1). With 5,000 products and random browsing these are chance co-occurrences, and lift is inflated because it divides by the individual product counts. The method is correct, but a minimum support (for example 20 co-occurrences) is needed before any recommendation is made; this dataset contains no real product affinity.

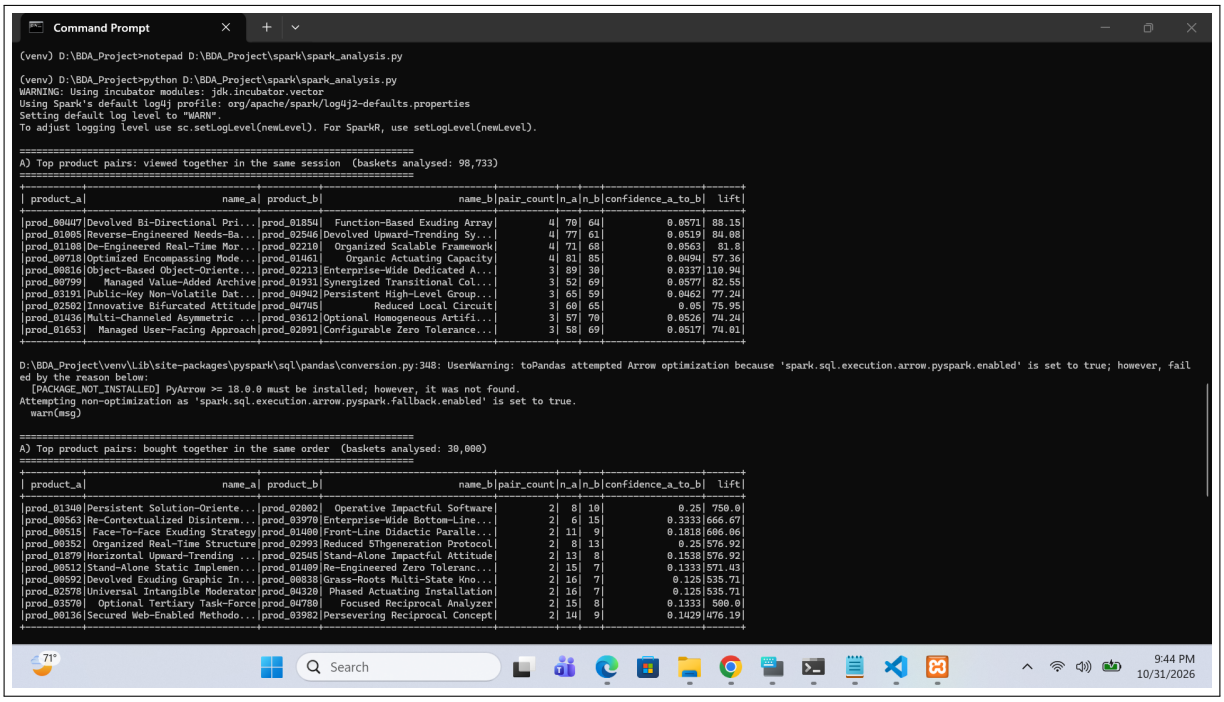


Figure 4.2-1: Top product pairs viewed together in a session and bought together in an order, with confidence and lift

The cohort analysis groups users by registration month (February to August 2026; the full cohorts hold 1,489 to 1,756 users) and follows their purchases by months since registration (Figure 4.2-2). Activity is flat: 60% to 66% of each full cohort buys in any month, and revenue per cohort user stays between about 1,030 and 1,185 a month, with no decay as the cohort ages, because the generator assigns purchases independently of registration.

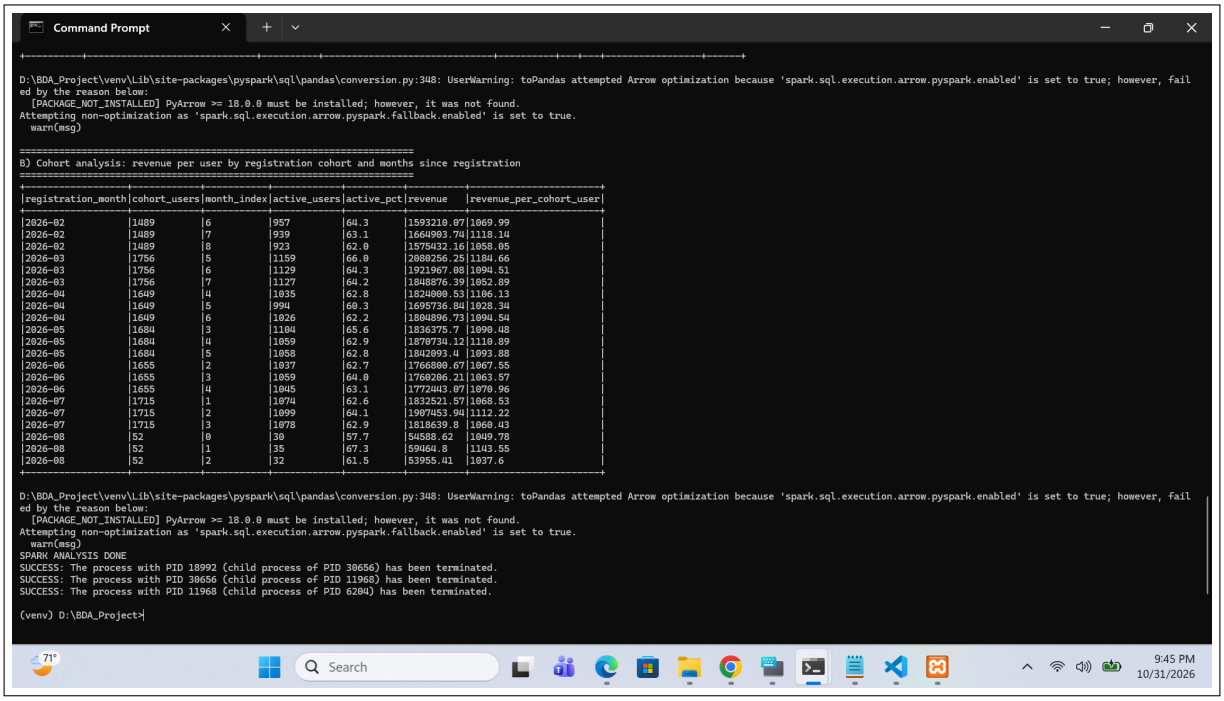


Figure 4.2-2: Cohort analysis: active users and revenue per cohort user by months since registration

4.3 Analyse with Spark SQL

Four queries run on the cleaned tables in `spark_sql.py`; the SQL text is printed above each result. **Q1** gives monthly revenue with month-over-month growth using a CTE and `LAG`: about 10,000 orders per month, an average order value of about 1,085, and revenue falling 1.0% and then 1.5% (Figure 4.3-1). The first month's growth is NULL because there is no previous month, and the first and last months are partial.

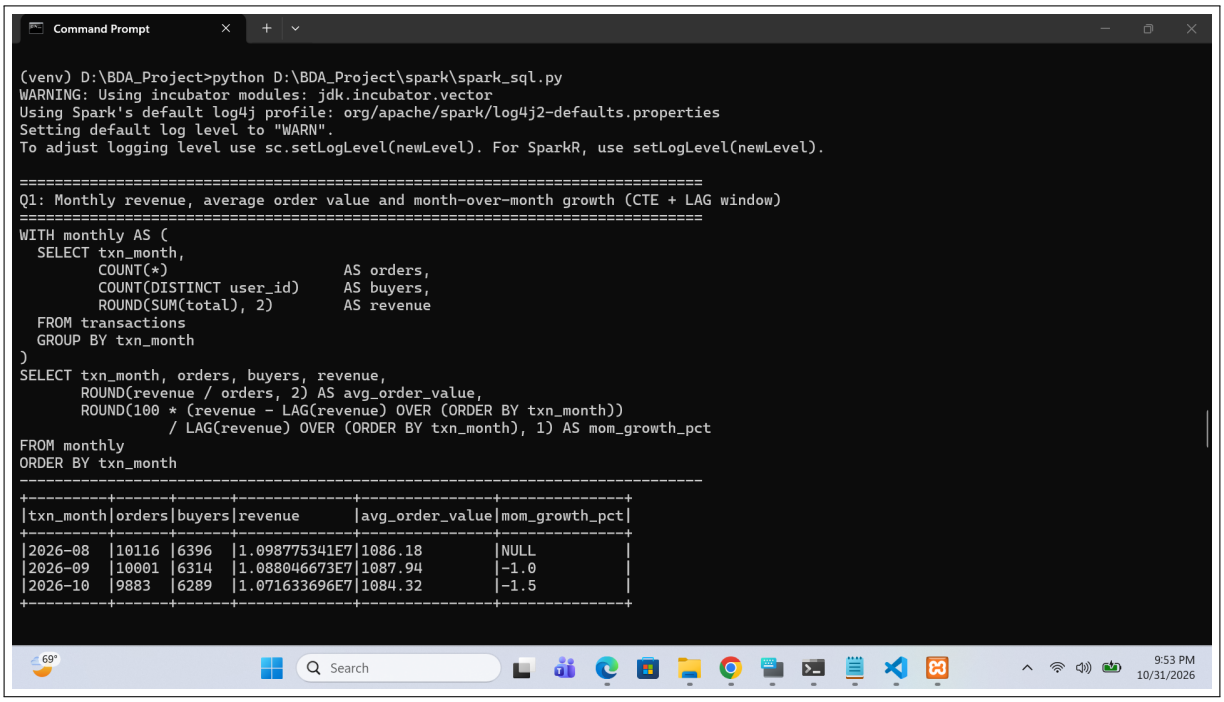


Figure 4.3-1: Q1: monthly revenue, average order value and growth (CTE and LAG)

Q2 ranks categories with window functions (Figure 4.3-2). The top category holds 4.78% of revenue and it takes 12 of 25 categories to reach half, so there is no Pareto concentration. Revenue and profit ranks differ: Kelly-Santiago is 1st in revenue and 15th in profit, Rodriguez-Graham 17th and 1st. These figures equal the MongoDB results of Section 2.3 (Kelly-Santiago 1,598,265.70 revenue and 329,242.73 estimated profit), so the two engines agree.

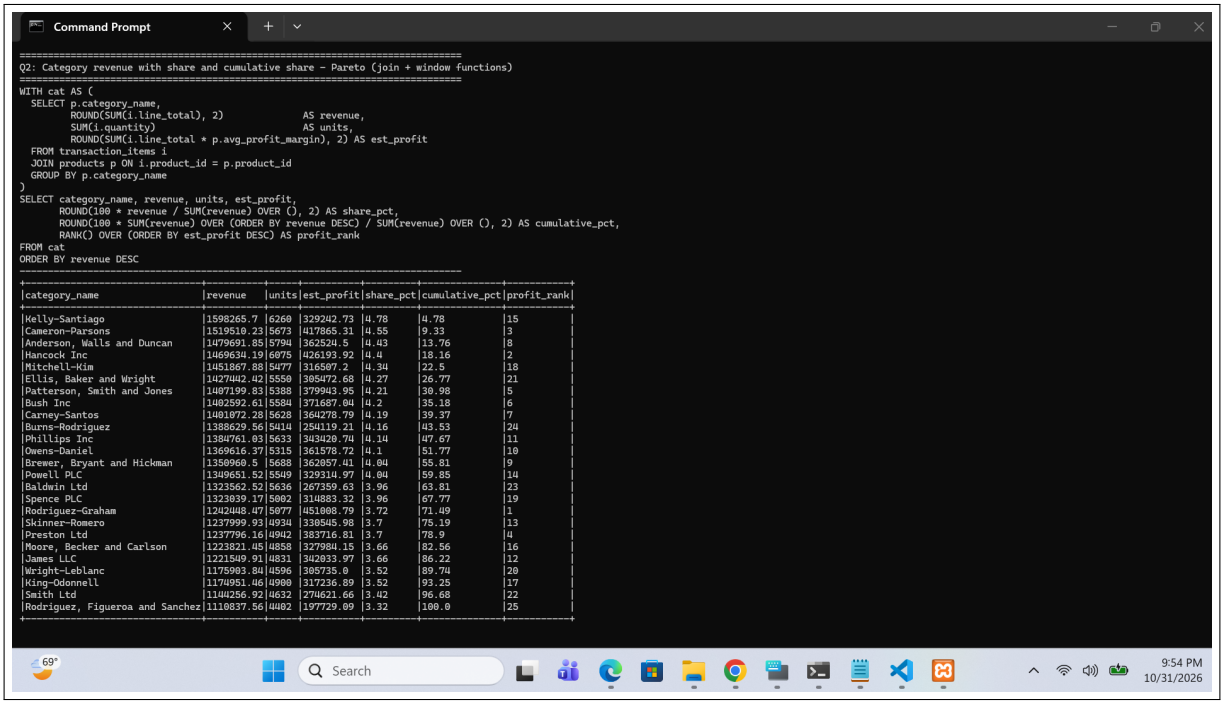


Figure 4.3-2: Q2: category revenue, share, cumulative share and profit rank (join and window functions)

Q3 compares conversion by traffic source and device (Figure 4.3-3). All 15 groups lie between 19.2% and 21.3%, a spread that is consistent with noise because the generator assigns source and device at random.

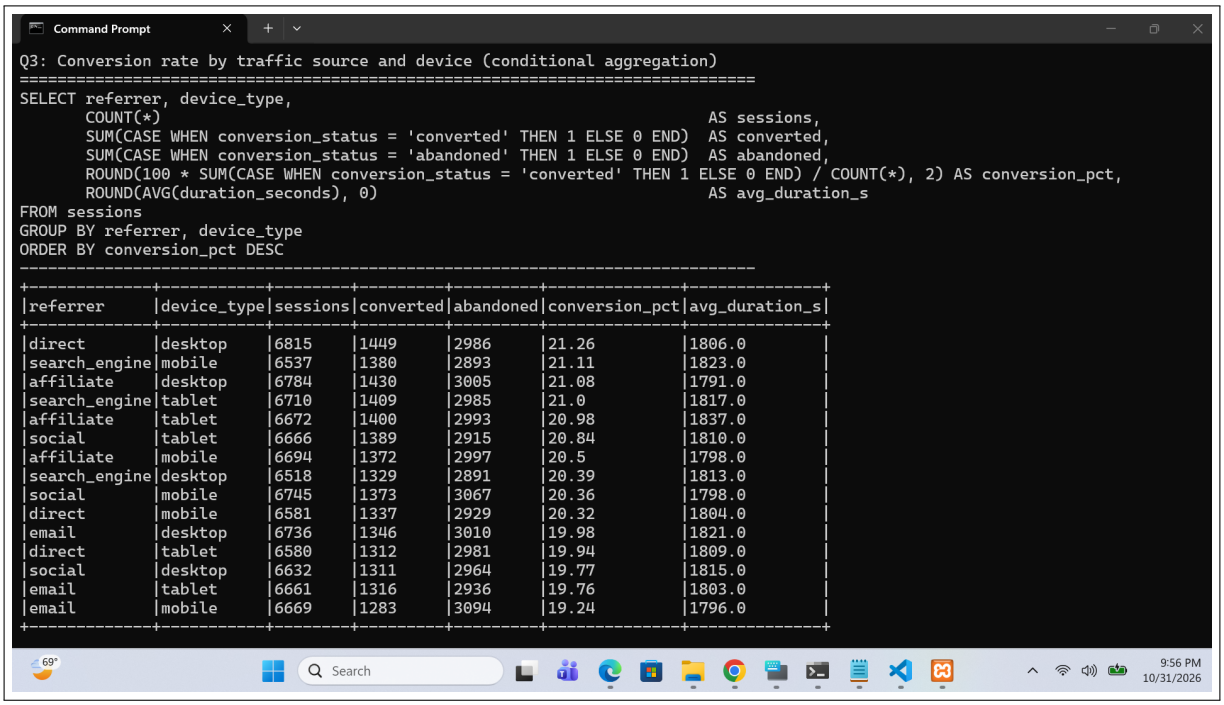


Figure 4.3-3: Q3: conversion rate by traffic source and device (conditional aggregation)

Q4 joins two views, `mongo_users` (profile and purchase totals, as held in MongoDB) and `hbase_user_sessions` (session activity, as held in HBase), to show how data from separate stores is combined in SQL (Figure 4.3-4). The top spender `user_007368` has 10 orders, 17,490.11 spent and 15 sessions, or 1,166 per session, while `user_009161` with 9 sessions spends 1,562.74 per session. Section 5 repeats this join with real extracts of both stores.

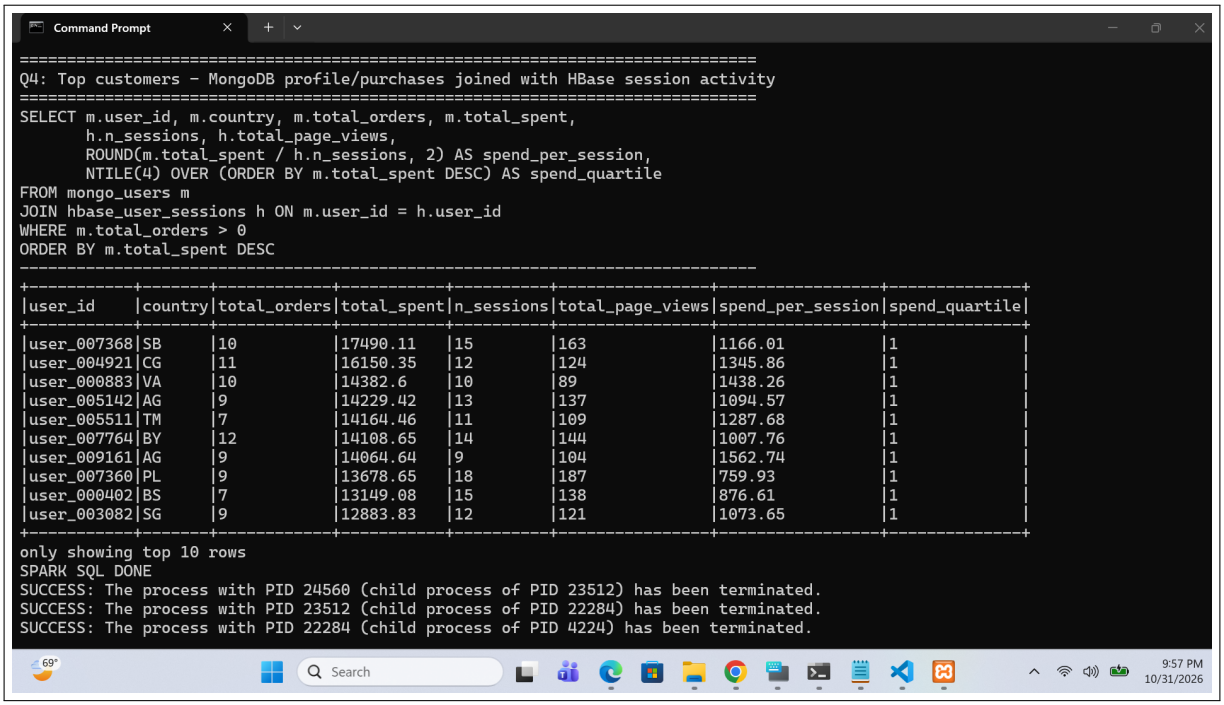


Figure 4.3-4: Q4: customers from the MongoDB-style view joined with the HBase-style view (join and `NTILE`)

5 Integrate MongoDB, HBase and Spark

Business question. Which customers are worth the most over the next 12 months, and does browsing engagement help identify them, including visitors who have not bought? The answer needs three systems: purchases and profiles live in MongoDB, session behaviour lives in HBase, and Spark joins them. Because 8 GB of RAM cannot run MongoDB, HBase and Spark together, each store was queried in turn, its result exported to CSV, and the two extracts joined in Spark.

5.1 Extract purchases and profiles from MongoDB

`export_from_mongo.py` reads the users collection and runs a four-stage aggregation inside MongoDB: `$unwind` the items, `$lookup` the product for its category margin, group by transaction, then group by user to get orders, net revenue, estimated profit and first and last purchase. Only 9,504 summary rows for the buyers leave MongoDB instead of 30,000 orders with their line items (Figure 5.1-1).

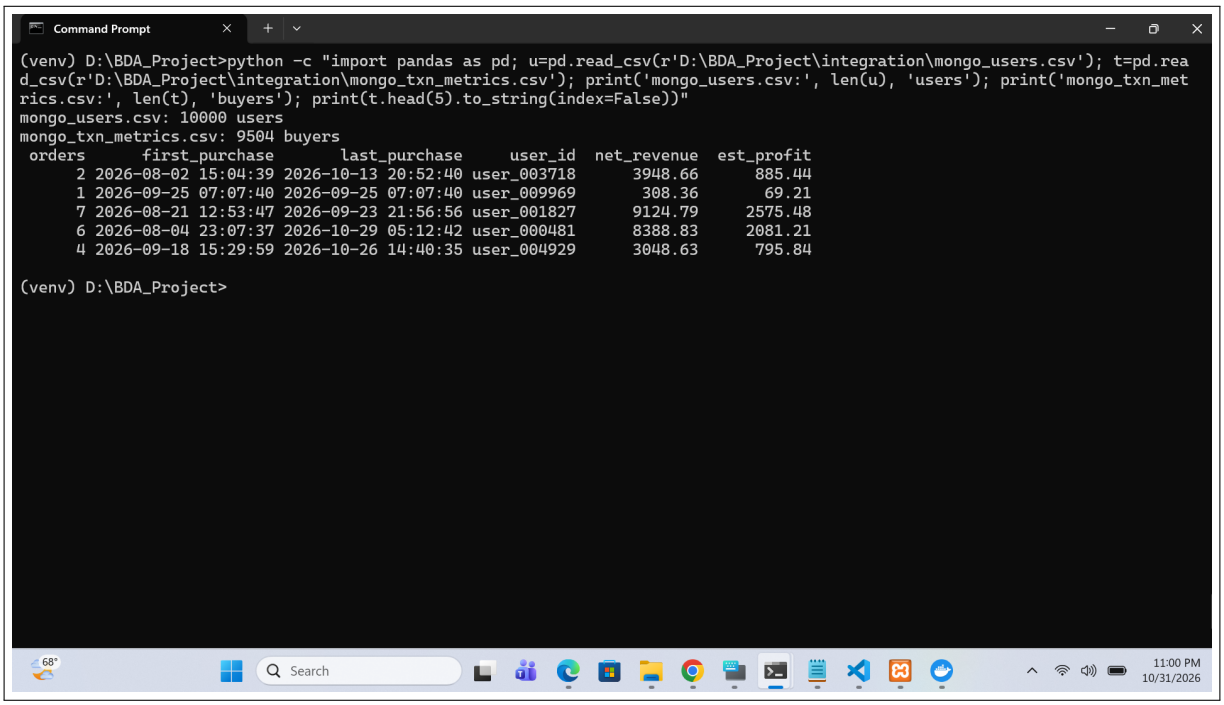


Figure 5.1-1: MongoDB extract: 10,000 users, 9,504 buyers and their purchase metrics

5.2 Extract session activity from HBase

`export_from_hbase.py` scans `user_sessions` for four columns from two families only, and rolls the 100,000 session rows up to sessions, total duration, page views and converted sessions per user, giving 10,000 users and 100,000 sessions in total (Figure 5.2-1).

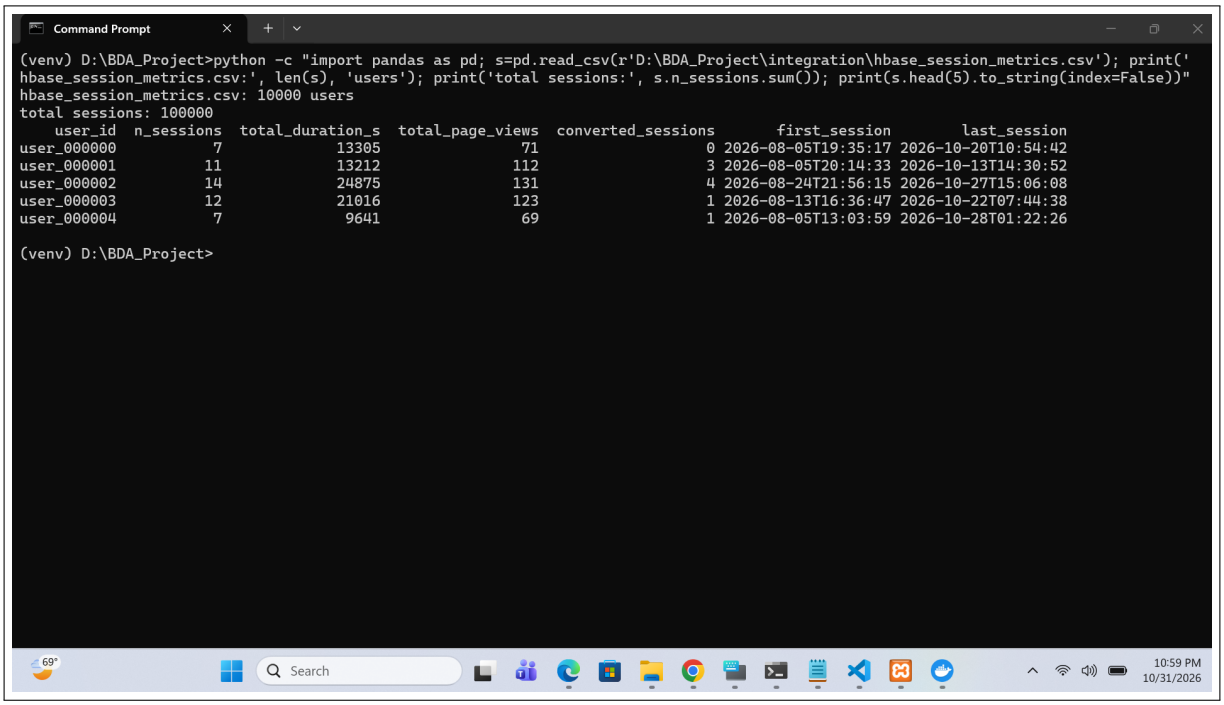


Figure 5.2-1: HBase extract: 100,000 sessions rolled up per user

5.3 Join and estimate lifetime value in Spark

`spark_clv.py` left-joins the users with the MongoDB purchase metrics and the HBase session metrics, so non-buyers are kept. Projected 12-month revenue is the revenue in the observed 91-day window multiplied by 365/91 (a factor of 4.01); projected profit uses the embedded category margin. Buyers are split into quartiles by projected revenue. As a check, the embedded `purchase_summary` in MongoDB equals the pipeline revenue for every user (0 mismatches, Figure 5.3-1).

Tier	Users	Avg orders	Avg order value	Projected 12-month revenue	Est. 12-month profit	Avg sessions
Platinum	2,376	5.00	1,333	25,602	6,614	10.75
Gold	2,376	3.48	1,206	15,508	4,011	10.19
Silver	2,376	2.53	1,096	9,752	2,536	9.82
Bronze	2,376	1.61	710	4,145	1,079	9.56
No purchase	496	0.00	-	0	0	8.49

Table 5-1: Customer lifetime value tiers

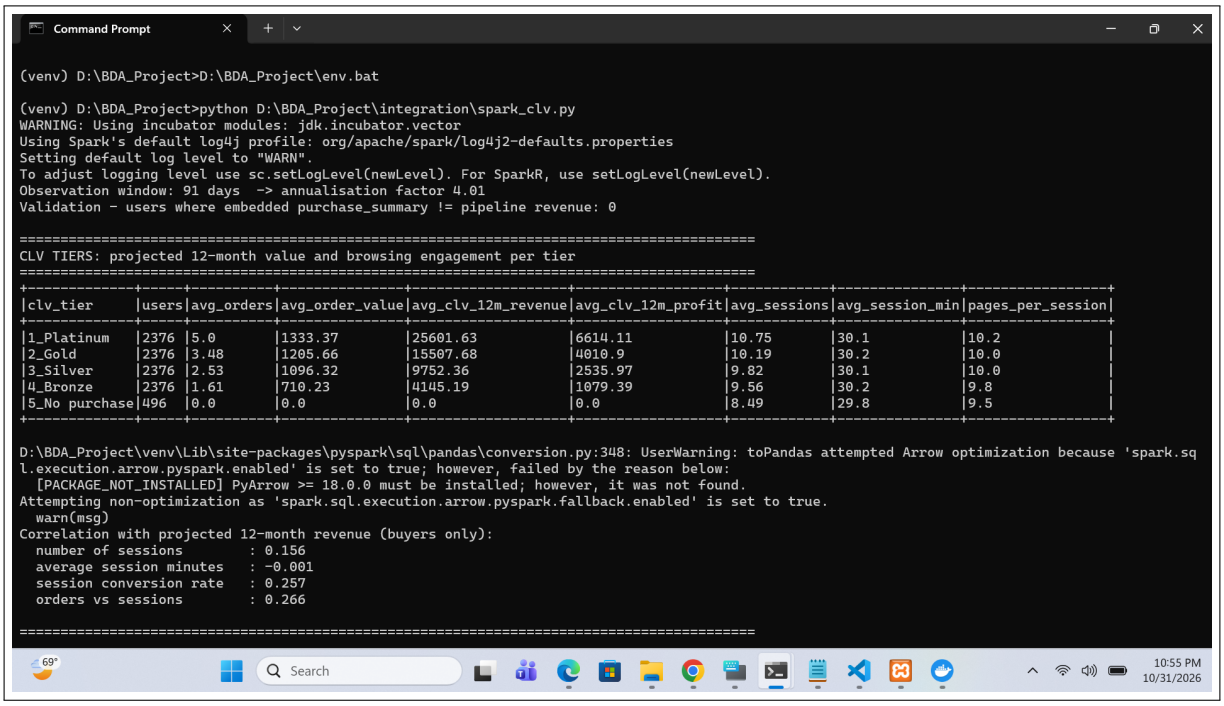


Figure 5.3-1: Spark CLV job: validation, CLV tiers and correlations with projected revenue

The top quarter of buyers holds about 46% of projected revenue and is worth 6.2 times the bottom quarter, while browsing barely separates the tiers: sessions correlate with projected revenue at 0.16, average minutes per session at -0.001, and orders with sessions at 0.27. The 0.26 correlation of session conversion rate is partly built in, because converted sessions create the orders. The list of heavy browsers is actionable: 76 users with at least 12 sessions (the top quartile) have never ordered, and the top customer `user_007368` is projected at 70,153 (Figure 5.3-2).

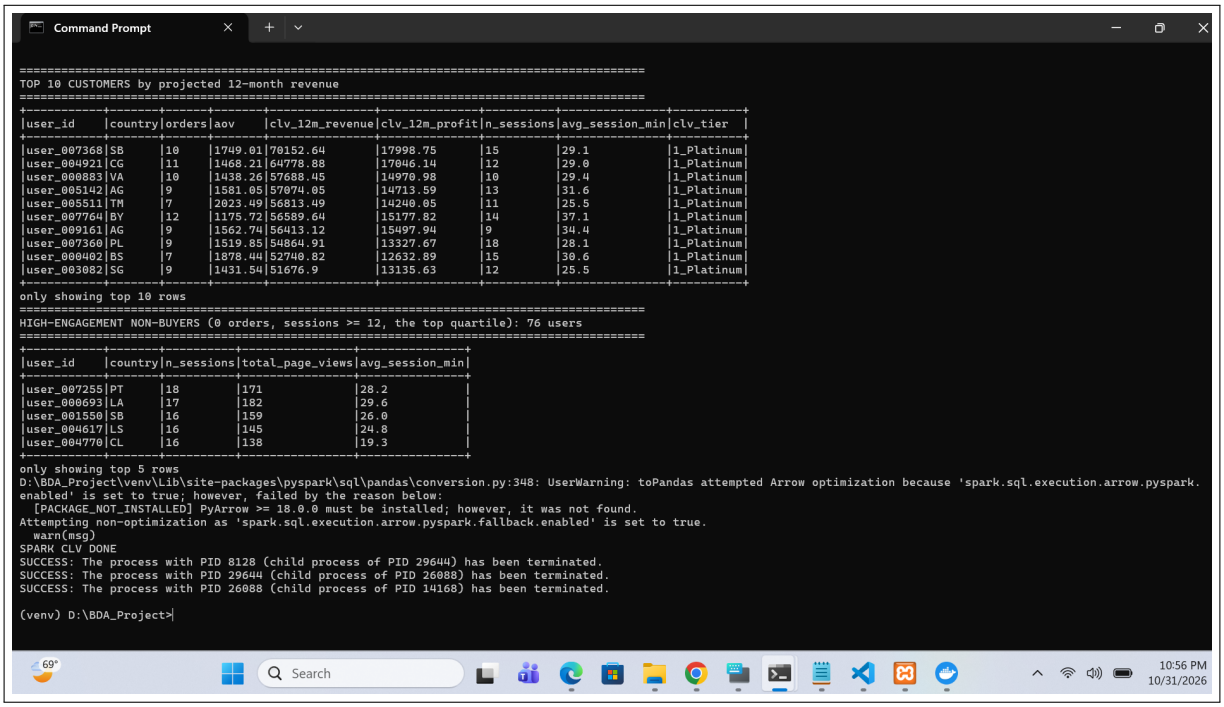


Figure 5.3-2: Top ten customers by projected 12-month revenue and the high-engagement non-buyers

Why these technologies. MongoDB computes the purchase metrics where the transactions live, HBase is scanned on four columns only, and Spark does the cross-system join and the feature computation on 10,000

rows instead of 130,000 source records. The extracts are snapshots, not live connections, which is stated as a limitation in Section 7.

6 Visualise and interpret the results

6.1 Conversion funnel

The funnel counts sessions that passed every earlier stage (Figure 6.1-1): of 100,000 sessions, 98,733 viewed a product, 65,082 added to the cart, 29,156 reached checkout and 20,436 purchased, an overall conversion of 20.4%. The largest loss is from cart to checkout, where only 44.8% continue, and 30% of the sessions that reach checkout do not pay (Figure 6.1-2).

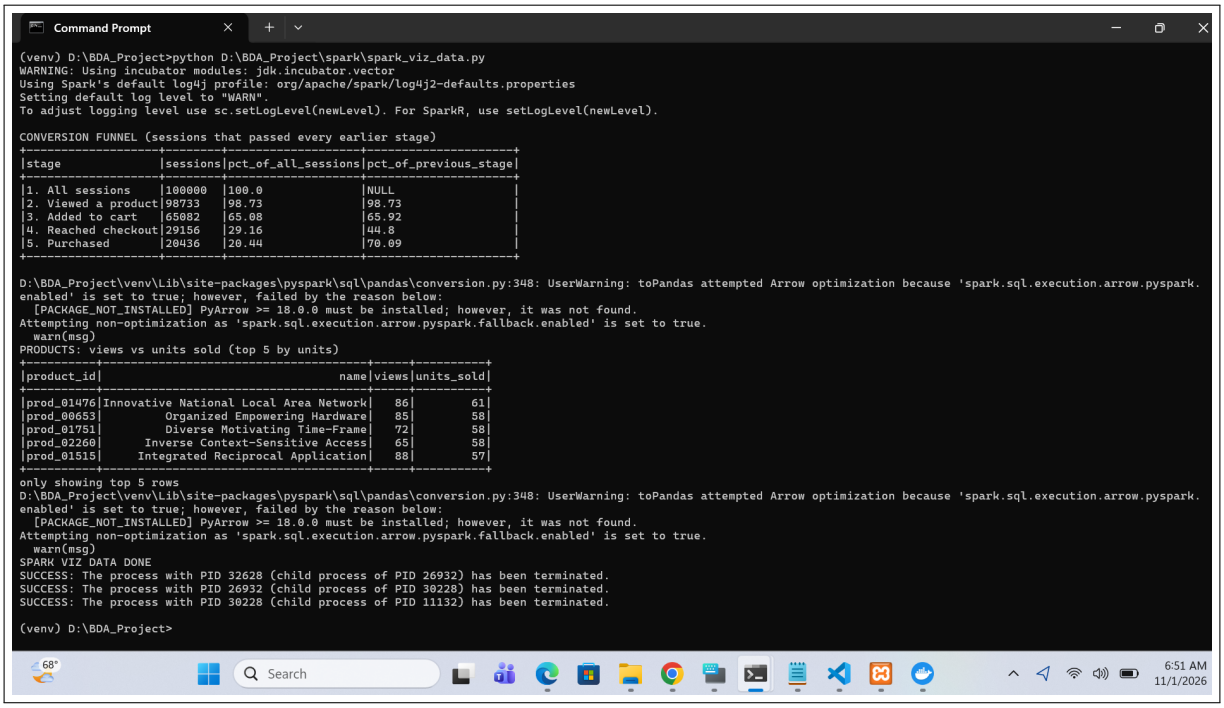


Figure 6.1-1: Conversion funnel table: sessions, share of all sessions and share of the previous stage

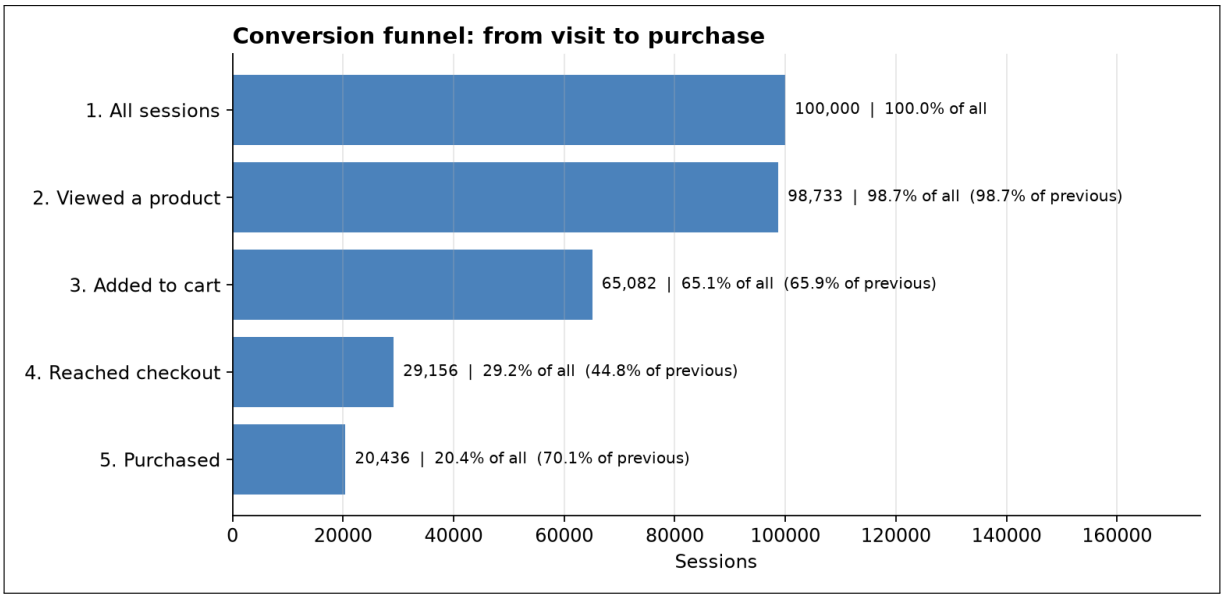


Figure 6.1-2: Conversion funnel: 20.4% of sessions purchase; checkout is the largest leak

6.2 Sales, customer value and categories

Weekly revenue is flat at roughly 2.4 to 2.6 million for every full week (Figure 6.2-1); the low first and last points are partial weeks at the ends of the 91-day window. Customer value differs about six-fold across tiers while the average number of sessions differs by only about 12% (Figure 6.2-2). Category revenue and estimated profit are only loosely related: Kelly-Santiago has the highest revenue but modest profit, and Rodriguez-Graham has the highest profit and the best margin, about 36% (Figure 6.2-3).

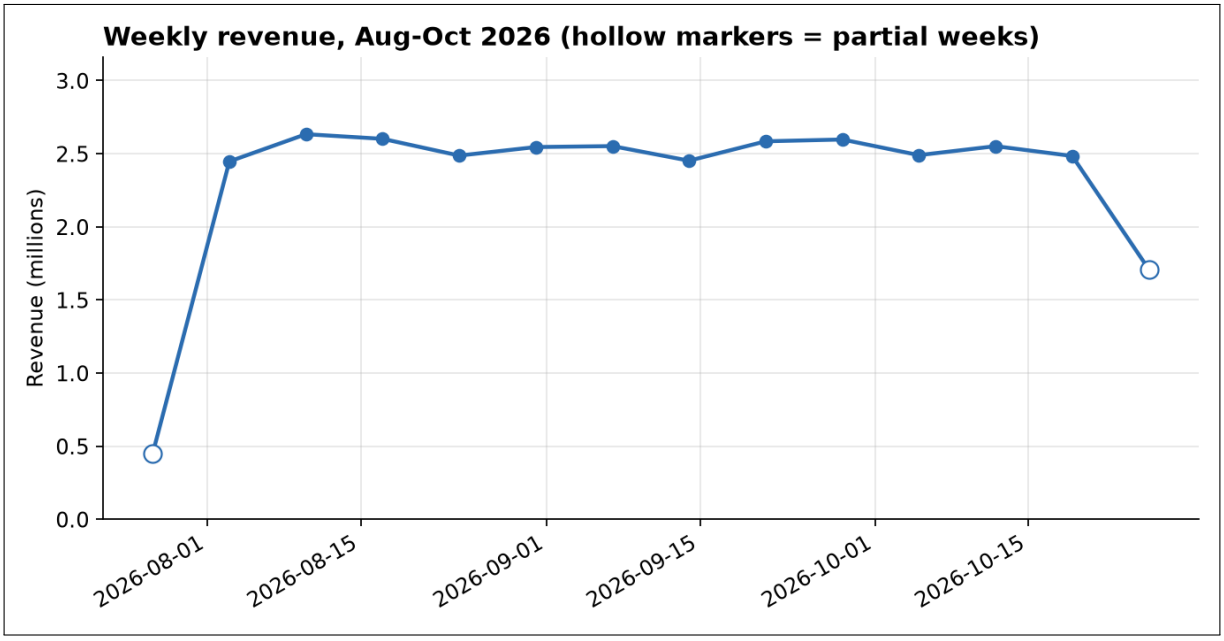


Figure 6.2-1: Weekly revenue, August to October 2026

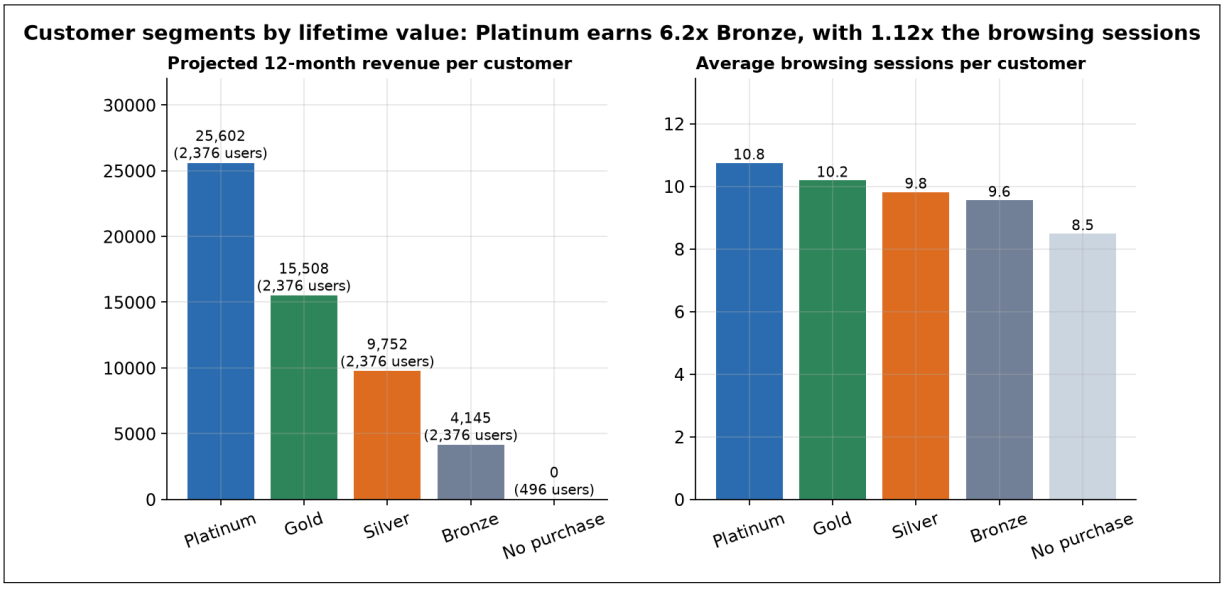


Figure 6.2-2: Customer tiers: projected 12-month revenue and average browsing sessions

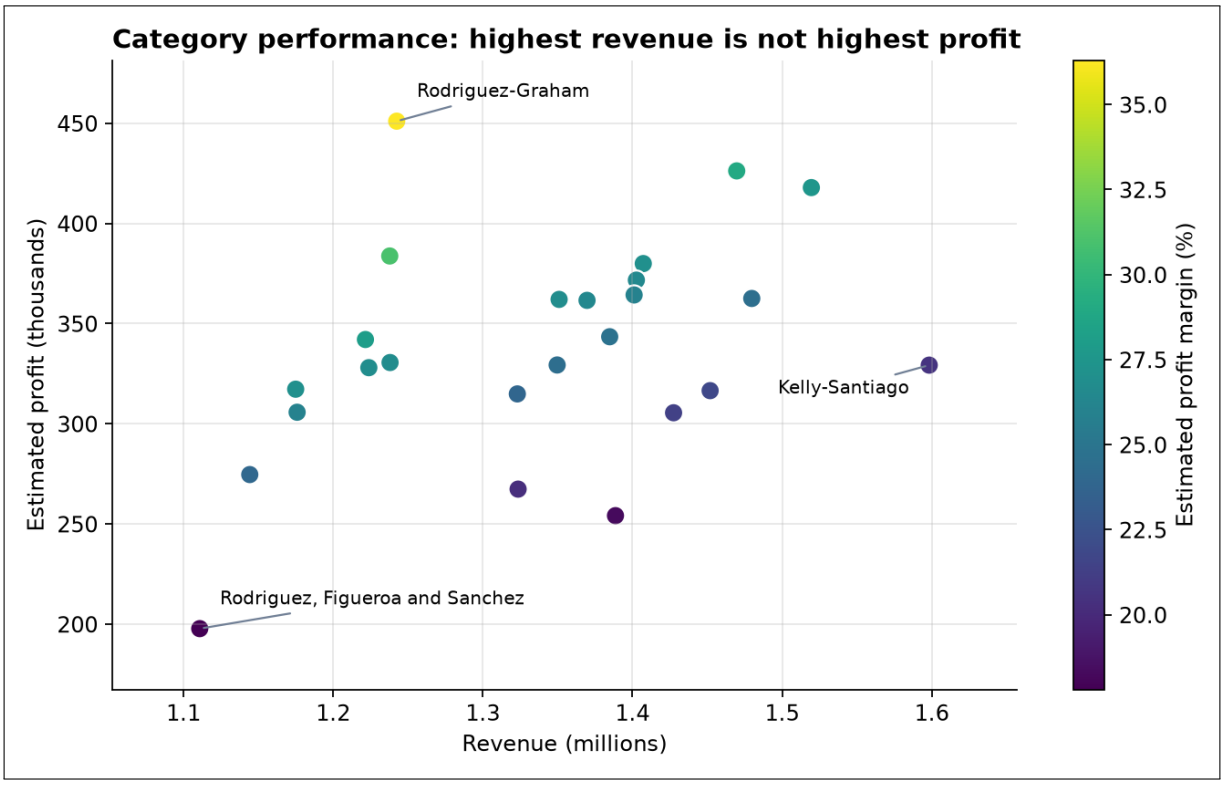


Figure 6.2-3: Category revenue against estimated profit, coloured by estimated margin

6.3 Business insights and actions

- **Checkout.** Test shipping-cost visibility and guest checkout at the cart-to-checkout step first, then payment friction, since 30% of checkouts are not completed.
- **Category profit.** Steer promotions and stock by estimated profit, not revenue; the two rankings differ strongly.
- **Customers.** Retarget the 76 heavy browsers who never bought, and protect the Platinum tier, which holds about 46% of projected revenue.
- **Engagement.** Time on site does not predict value here, so engagement should not be used alone to rank customers.

7 Scale and defend the design

7.1 Scalability

- **MongoDB.** Shard transactions and sessions on a hashed `user_id`; keep embedded documents bounded (page views stay inside one session); keep margins embedded to avoid `$lookup` at scale.
- **HBase.** The `user_id` prefix spreads writes across regions while the reversed timestamp keeps each user's history together; pre-split regions by user range and read only the needed families.
- **Spark.** The same code runs on a cluster by changing the master and memory; Parquet allows column pruning and date partitioning; small tables such as products can be broadcast in joins.
- **Integration.** Replace CSV snapshots with the MongoDB and HBase connectors, schedule the CLV job, and stream sessions into HBase with incremental recomputation.

7.2 Technology selection

Task	Chosen	Why	Alternative and why not
Transactions and profiles	MongoDB	Nested documents and flexible aggregation	HBase has no secondary indexes or aggregation framework
Session history by user and time	HBase	Row key gives newest-first scans by construction	MongoDB needs an index and a sort for the same result
Cleaning, SQL, joins across stores	Spark	Distributed, SQL and DataFrames, Parquet	Python loops would not scale

Table 7-1: Which technology serves which task, and the alternative that was rejected

7.3 Limitations and future work

- The generator assigns sessions, purchases, source and device at random, so conversion by channel, cohort decay and engagement against value are not realistic; the pipelines, not the patterns, are what is demonstrated.
- Products have no `subcategory_id`; user `last_active` is inconsistent with purchases; some transactions have no session, so purchases can exceed views.
- CLV extrapolates 91 days linearly (no churn or seasonality), ignores discounts in the profit estimate, and uses snapshot extracts; the project ran on 8 GB of RAM.
- Future work: real behavioural data, a purchase-probability model in the CLV, minimum-support recommendations, live connectors and streaming ingestion.

8 Submit reproducible work

8.1 Repository

All code, queries, result files, charts and screenshots are in the GitHub repository, and its README lists the commands in execution order. Raw generated data and database stores are not uploaded; the README explains how to regenerate them.

Repository URL: `https://github.com/erneste-uwimpuhwe/101414_BDA_FINAL_EXAM_PROJECT_SUBMISSION`

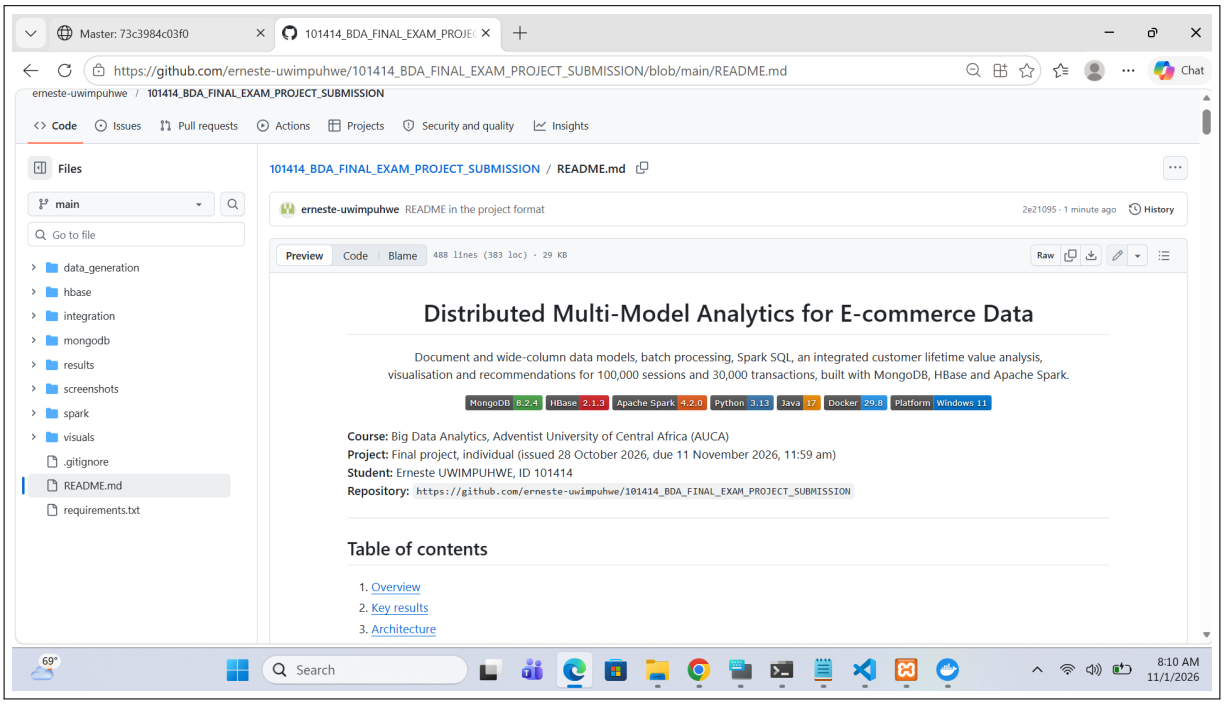


Figure 8.1-1: The GitHub repository: file tree and the rendered README with title, badges, course and repository lines, and table of contents

8.2 Reproduce the results

- Generate the data: `data_generation/dataset_generator.py`.
- MongoDB: start `mongod` on port 27018, then `load_mongo.py` and `mongo_aggregations.py`.
- HBase: start the Docker container, run `create_tables.hbase`, `hbase_load.py`, `hbase_queries.py` and `hbase_shell_queries.hbase`.
- Spark: `prep_sessions_jsonl.py`, `spark_clean.py`, `spark_analysis.py` and `spark_sql.py` (Docker stopped to free memory).
- Integration: `export_from_mongo.py`, `export_from_hbase.py`, then `spark_clv.py`.
- Charts: `spark_viz_data.py` and `make_charts.py`.

After the work, the HBase container and `mongod` were stopped to release memory; the data remain on drive D:.

8.3 Academic integrity

The supplied dataset generator is the only code provided with the course. Its generation logic is unchanged; only the target volumes were set to 100,000 sessions and 30,000 transactions. All loading, query, Spark, integration and chart code was run and checked by the author, and the results were cross-checked between MongoDB and Spark. An AI assistant (Claude, from Anthropic) was used for guidance and to help draft code and documentation; every figure in this report comes from running the programs.